

Compre Notes for DIP

Pranav

December 12, 2025

Contents

1	Convolution	6
1.1	Theory	6
1.2	Mathematical Formulation	6
1.3	Example 1: Smoothing by Convolution	7
1.4	Example 2: Edge Detection by Convolution	7
1.5	Summary	7
2	Filtering	8
2.1	Theory	8
2.2	Mathematical Formulation	8
2.2.1	Linear Filtering	8
2.2.2	Nonlinear Filtering	9
2.3	Important Types of Filters	9
2.3.1	Smoothing (Low-Pass) Filters	9
2.3.2	Sharpening (High-Pass) Filters	9
2.3.3	Bandpass and Bandreject Filters	10
2.4	Example Problem 1: Gaussian Smoothing	10
2.5	Example Problem 2: Median Filter for Impulse Noise	11
3	Common Filters Used in Image Processing	11
3.1	Smoothing (Low-Pass) Filters	11
3.1.1	Averaging Filter (Mean Filter)	11
3.1.2	Weighted Averaging Filter	12
3.1.3	Gaussian Filter	12
3.2	Sharpening (High-Pass) Filters	12
3.2.1	Laplacian Filter	12
3.2.2	Sharpening via Unsharp Masking	12
3.2.3	High-Boost Filtering	13
3.3	Gradient-Based Filters	13
3.3.1	Sobel Filters	13
3.3.2	Prewitt Filters	13
3.4	Nonlinear Filters	13

3.4.1	Median Filter	13
3.4.2	Max Filter	14
3.4.3	Min Filter	14
3.4.4	Midpoint Filter	14
3.5	Bandpass and Bandreject Filters (Spatial Approximations)	14
3.5.1	High-Pass Approximation	14
3.5.2	Low-Pass Approximation	14
3.5.3	Bandpass	14
3.6	Directional Filters	15
3.6.1	Horizontal Line Detector	15
3.6.2	Vertical Line Detector	15
3.7	Summary	15
3.8	Summary	15
4	Fourier Transform	15
4.1	Theory	15
4.2	Mathematical Formulation	16
4.2.1	Continuous 2D Fourier Transform	16
4.2.2	Discrete 2D Fourier Transform (DFT)	16
4.3	Key Properties of the Fourier Transform	17
4.3.1	Linearity	17
4.3.2	Shift Property	17
4.3.3	Convolution Theorem	17
4.3.4	Correlation Theorem	17
4.3.5	Separability	17
4.3.6	Symmetry for Real Images	17
4.3.7	Centering the Fourier Transform	17
4.4	Interpretation in Image Processing	18
4.5	Example Problem 1: Fourier Transform of a Separable Image	18
4.6	Example Problem 2: Effect of Image Shift on Fourier Transform	19
4.7	Summary	19
5	Inverse Fourier Transform	20
5.1	Theory	20
5.2	Mathematical Formulation	20
5.2.1	Continuous 2D Inverse Fourier Transform	20
5.2.2	Discrete 2D Inverse Fourier Transform (IDFT)	20
5.3	Properties of the Inverse Fourier Transform	21
5.3.1	1. Duality	21
5.3.2	2. Uniqueness	21
5.3.3	3. Hermitian Symmetry	21
5.3.4	4. Energy Conservation (Parseval's Theorem)	21
5.3.5	5. Circular Convolution	21
5.4	Interpretation in Image Processing	21
5.5	Example Problem 1: Recovering Spatial Image from Simple Spectrum	22

5.6	Example Problem 2: IFT After Filtering	22
5.7	Summary	23
6	Quantization	23
6.1	Theory	23
6.2	Mathematical Formulation	24
6.2.1	Quantization Error	24
6.2.2	Signal-to-Noise Ratio (SNR)	24
6.3	Types of Quantizers	25
6.3.1	Uniform Quantization	25
6.3.2	Non-Uniform Quantization	25
6.4	Effects of Quantization in Imaging	25
6.4.1	False Contouring	25
6.4.2	Loss of Detail	25
6.4.3	Quantization Noise in Smooth Areas	25
6.5	Example Problem 1: Quantization Error and SNR	26
6.6	Example Problem 2: False Contouring Analysis	26
6.7	Summary	27
7	Histogram Equalization	27
7.1	Theory	27
7.2	Mathematical Derivation (Based on Gonzalez and Woods)	28
7.2.1	1. Transformation Function	28
7.2.2	2. Why Histogram Equalization Works	28
7.3	Algorithmic Steps (Discrete Implementation)	29
7.4	Interpretation	29
7.5	Local Histogram Equalization (Adaptive HE and CLAHE)	29
7.5.1	Local HE	29
7.5.2	CLAHE (Contrast Limited AHE)	30
7.6	Example Problem 1: Manual Histogram Equalization	30
7.7	Example Problem 2: Histogram Equalization on a Low-Contrast Image	31
7.8	Summary	32
8	Degradation and the Point Spread Function (PSF)	32
8.1	Introduction	32
8.2	Degradation Model	32
8.3	Point Spread Function (PSF)	33
8.3.1	Properties of PSF	33
8.4	Common PSF Models in Imaging	33
8.4.1	1. Motion Blur PSF (Uniform Linear Motion)	33
8.4.2	2. Out-of-Focus Blur (Defocus Blur)	34
8.4.3	3. Atmospheric Turbulence Blur	34
8.4.4	4. Gaussian Blur PSF	34
8.5	Noise in the Degradation Model	34
8.6	Why PSF Matters in Restoration	35
8.7	Example Problem 1: Motion Blur PSF and OTF	35

8.8	Example Problem 2: Gaussian PSF Degradation	36
8.9	Summary	36
9	Discrete Cosine Transform (DCT)	37
9.1	Introduction	37
9.2	1D DCT Definition	37
9.3	2D DCT (Most Relevant for Images)	37
9.4	Interpretation of DCT Coefficients	38
9.5	Why the DCT Compacts Energy Well	38
9.6	DCT in JPEG Compression	38
9.7	Example Problem 1: Compute DCT of a Simple 1D Signal	39
9.8	Example Problem 2: 2D DCT of an 2×2 Block	39
9.9	Summary	40
10	Text-Based Data Compression	40
10.1	Introduction	40
10.2	Types of Redundancy	40
10.3	Information Theory Basics	41
10.3.1	Entropy	41
10.3.2	Average Codeword Length	41
10.4	Prefix Codes	41
10.5	Lossless vs. Lossy Compression	42
10.5.1	Lossless Compression	42
10.5.2	Lossy Compression	42
10.6	Modeling Data for Compression	42
10.7	Two-Part Structure of Compression Algorithms	42
10.8	Example Problem 1: Compute Entropy and Minimum Bits Required	43
10.9	Example Problem 2: Verify Kraft Inequality and Prefix Condition	43
10.10	Summary	44
11	LZ77 Compression	44
11.1	Introduction	44
11.2	Sliding Window Model	45
11.3	Matching Procedure	45
11.4	Decoding	45
11.5	Why LZ77 Works	46
11.6	LZ77 in Image Processing	46
11.7	Example Problem 1: Manual LZ77 Encoding	46
11.8	Example Problem 2: LZ77 Decoding	47
11.9	Summary	48
12	LZ78 Compression	48
12.1	Introduction	48
12.2	Encoding Mechanism	48
12.2.1	Encoding Steps	49
12.3	Decoding	49

12.4	Comparison: LZ77 vs. LZ78	49
12.5	Example Problem 1: LZ78 Encoding	49
12.6	Example Problem 2: LZ78 Decoding	50
12.7	Summary	51
13	LZW Compression	51
13.1	Introduction	51
13.2	Key Idea	52
13.3	Encoding Algorithm	52
13.4	Decoding Algorithm	52
13.5	Why LZW Works Well	53
13.6	LZW in Image Processing (GIF/TIFF)	53
13.7	Example Problem 1: LZW Encoding	53
13.8	Example Problem 2: LZW Decoding	54
13.9	Summary	54
14	Encoding Theory	55
14.1	Introduction	55
14.2	Uniquely Decodable and Prefix Codes	55
14.3	Kraft–McMillan Inequality	56
14.4	Optimality of Prefix Codes	56
14.5	Entropy and Average Code Length	56
14.6	Instantaneous Codes	56
14.7	Canonical Codes	57
14.8	Example Problem 1: Checking Prefix Validity	57
14.9	Example Problem 2: Designing a Prefix Code From Given Lengths	58
14.10	Summary	58
15	Huffman Coding	59
15.1	Introduction	59
15.2	Constructing the Huffman Code	59
15.3	Properties of Huffman Codes	59
15.4	Canonical Huffman Coding (Used in PNG/ZIP)	60
15.5	Example Problem 1: Full Huffman Construction	60
15.6	Example Problem 2: Canonical Huffman Code Construction	61
15.7	Summary	62
16	Arithmetic Coding	62
16.1	Introduction	62
16.2	Basic Concept	63
16.3	Encoding Procedure	63
16.4	Decoding Procedure	64
16.5	Advantages Over Huffman Coding	64
16.6	Renormalization (Practical Implementation)	64
16.7	Example Problem 1: Arithmetic Encoding	65
16.8	Example Problem 2: Arithmetic Decoding	66

16.9 Summary	67
17 Golomb–Rice Coding	67
17.1 Introduction	67
17.2 Golomb Coding Overview	67
17.3 Truncated Binary Encoding of the Remainder	68
17.4 Decoding	68
17.5 Choosing the Parameter m	68
17.6 Rice Coding (Special Case)	68
17.7 Example Problem 1: Golomb Encoding	69
17.8 Example Problem 2: Rice Coding	69
17.9 Why Golomb–Rice Coding Works for Images	70
17.10 Summary	70
18 Haar Wavelet Transform	70
18.1 Introduction	70
18.2 Haar Scaling and Wavelet Functions	71
18.3 1D Haar Transform	71
18.4 Inverse 1D Haar Transform	71
18.5 2D Haar Transform (Images)	72
18.6 Wavelet-Based Image Compression	72
19 Wavelet Compression Using the LL Band	76
19.1 Introduction	76
19.2 Energy Compaction Property	77
19.3 Compression Pipeline	77
19.4 Multi-Level LL Decomposition	77
19.5 Rate–Distortion Considerations	78
19.6 Compression Ratio Calculation	78

1 Convolution

1.1 Theory

Convolution is a fundamental operation in linear image processing, describing how a spatial filter (kernel) modifies an image. It represents the weighted sum of pixel intensities within a neighborhood, where the weights are given by the filter coefficients.

Formally, convolution models the response of a linear, shift-invariant (LSI) system when an image $f(x, y)$ passes through an impulse response $h(x, y)$.

1.2 Mathematical Formulation

For continuous images:

$$g(x, y) = (f * h)(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(\xi, \eta) h(x - \xi, y - \eta) d\xi d\eta \quad (1)$$

For discrete digital images:

$$g[m, n] = \sum_{s=-a}^a \sum_{t=-b}^b f[m-s, n-t] h[s, t] \quad (2)$$

where $h[s, t]$ is the filter mask of size $(2a + 1) \times (2b + 1)$.

Properties:

- **Commutative:** $f * h = h * f$
- **Associative:** $(f * h_1) * h_2 = f * (h_1 * h_2)$
- **Distributive:** $f * (h_1 + h_2) = f * h_1 + f * h_2$
- **Shift Invariance:** Translating the input shifts the output by the same amount.

Intuitive Meaning: Each output pixel $g[m, n]$ is computed as a local weighted average of input pixels, centered at (m, n) , using the flipped kernel $h[-s, -t]$.

1.3 Example 1: Smoothing by Convolution

Let

$$f = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad h = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Then

$$g(2, 2) = \frac{1}{9} \sum_{i=-1}^1 \sum_{j=-1}^1 f(2-i, 2-j) = \frac{1}{9}(1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9) = 5$$

Thus, each pixel becomes the average of its 3×3 neighborhood. This operation smooths the image and suppresses noise, corresponding to a low-pass filter in the frequency domain.

1.4 Example 2: Edge Detection by Convolution

Consider the Sobel horizontal edge detector:

$$h = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

When applied to an image, the convolution emphasizes regions with large horizontal intensity gradients.

For a row $f = [10, 20, 40]$, the convolution result (1D simplification) is:

$$g = (-1)(10) + (0)(20) + (1)(40) = 30$$

A large magnitude indicates a strong edge at that position.

1.5 Summary

Convolution is a mathematical model for spatial filtering that:

- Describes linear, shift-invariant image transformations.
- Links directly to multiplication in the frequency domain via the Convolution Theorem.
- Serves as the computational foundation for blurring, sharpening, and noise removal operations.

2 Filtering

2.1 Theory

Filtering refers to the process of modifying or enhancing an image by operating on its pixel values using a neighborhood-based operator. Filters can be classified broadly into:

- **Linear filters:** The output is a weighted sum of input pixels (e.g., averaging, Gaussian).
- **Nonlinear filters:** The output is computed via nonlinear operations (e.g., median, max/min).

In image processing, filtering serves multiple purposes:

- Noise reduction (smoothing)
- Edge enhancement (sharpening)
- Feature extraction (edge/line detection)

- Texture analysis
- Preprocessing for segmentation or compression

Filtering is implemented either in the **spatial domain** (mask-based operations) or the **frequency domain** (multiplication with a frequency response).

2.2 Mathematical Formulation

2.2.1 Linear Filtering

A linear spatial filter computes each output pixel as:

$$g[m, n] = \sum_{s=-a}^a \sum_{t=-b}^b w[s, t] f[m - s, n - t] \quad (3)$$

where $w[s, t]$ is the filter mask and $(2a + 1) \times (2b + 1)$ is its size.

This is equivalent to convolution:

$$g = f * w$$

Examples include:

- Averaging filter
- Gaussian smoothing filter
- Laplacian sharpening
- Sobel/Prewitt gradient filters

2.2.2 Nonlinear Filtering

Nonlinear filters do not compute weighted sums. Instead, they use ranking or other nonlinear rules:

$$g[m, n] = \text{nonlinear}(f \text{ in neighborhood})$$

Examples:

- **Median filter:**

$$g[m, n] = \text{median}\{f[i, j] \mid (i, j) \in \mathcal{N}_{m,n}\}$$

- Min/max filtering
- Order-statistic filters

Nonlinear filters are especially useful for removing **salt-and-pepper noise**.

2.3 Important Types of Filters

2.3.1 Smoothing (Low-Pass) Filters

Used to reduce high-frequency noise.

Averaging Filter:

$$w = \frac{1}{mn} \mathbf{1}_{m \times n}$$

Gaussian Low-Pass Filter:

$$w(s, t) = \frac{1}{2\pi\sigma^2} e^{-\frac{s^2+t^2}{2\sigma^2}}$$

This filter preserves edges better than simple averaging.

2.3.2 Sharpening (High-Pass) Filters

Highlight high-frequency content such as edges.

Laplacian Filter:

$$\nabla^2 f = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} * f$$

Sharpened image:

$$g = f - \lambda(\nabla^2 f)$$

Gradient Filters (Sobel):

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

2.3.3 Bandpass and Bandreject Filters

Used to isolate or remove specific frequency ranges. Mathematically defined in the frequency domain as:

$$G(u, v) = H(u, v)F(u, v)$$

Where $H(u, v)$ selectively passes or rejects frequency bands.

2.4 Example Problem 1: Gaussian Smoothing

Problem

Given the 3×3 Gaussian filter with $\sigma = 1$:

$$w = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Apply the filter to the 3×3 image:

$$f = \begin{bmatrix} 10 & 20 & 30 \\ 20 & 40 & 60 \\ 30 & 60 & 90 \end{bmatrix}$$

Compute the filtered output at the center pixel.

Solution

$$g(2, 2) = \frac{1}{16}(1 \cdot 10 + 2 \cdot 20 + 1 \cdot 30 + 2 \cdot 20 + 4 \cdot 40 + 2 \cdot 60 + 1 \cdot 30 + 2 \cdot 60 + 1 \cdot 90)$$

Compute numerator:

$$10 + 40 + 30 + 40 + 160 + 120 + 30 + 120 + 90 = 640$$

Thus:

$$g(2, 2) = \frac{640}{16} = 40$$

Interpretation

The Gaussian filter smooths the data while preserving general brightness structure; the center pixel becomes a high-confidence weighted local average.

2.5 Example Problem 2: Median Filter for Impulse Noise

Problem

Apply the 3×3 median filter to the following neighborhood:

$$\{200, 205, 0, 198, 202, 210, 0, 199, 203\}$$

Assume the zeros are salt-and-pepper noise. Compute the filtered output.

Solution

Sort the values:

$$0, 0, 198, 199, 200, 202, 203, 205, 210$$

Median = 200.

Interpretation

The median filter removes impulse noise while preserving edges, unlike averaging filters which would smear pixel intensities.

3 Common Filters Used in Image Processing

This section summarizes the most important spatial-domain filters used in digital image processing. For each filter, we provide:

- The purpose and intuition
- Mathematical/matrix form
- Image-processing-specific usage

All filters operate on local neighborhoods (usually 3×3 or 5×5).

3.1 Smoothing (Low-Pass) Filters

3.1.1 Averaging Filter (Mean Filter)

Reduces noise by computing the mean of pixel intensities.

3×3 Mask:

$$H = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Usage: Noise reduction, rough smoothing. Blurs edges significantly.

3.1.2 Weighted Averaging Filter

Gives more weight to central pixels.

$$H = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Usage: Better smoothing with less edge distortion than the simple mean filter.

3.1.3 Gaussian Filter

Approximates a Gaussian blur, optimal for noise reduction without ringing.

$$H = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

(This is a discrete approximation for $\sigma = 1$.)

Usage: Reduces Gaussian noise; preserves edge contours better than averaging.

3.2 Sharpening (High-Pass) Filters

Sharpening filters enhance edges by emphasizing high-frequency components.

3.2.1 Laplacian Filter

Isotropic second-derivative operator.

4-neighbor version:

$$H = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

8-neighbor version:

$$H = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Usage: Edge enhancement, image sharpening, feature extraction.

3.2.2 Sharpening via Unsharp Masking

$$g = f - f_{LP}$$

3.2.3 High-Boost Filtering

$$g = Af - f_{LP}, \quad A > 1$$

Matrix-based implementation uses any smoothing filter for f_{LP} .

3.3 Gradient-Based Filters

Approximations to the first derivative for edge detection.

3.3.1 Sobel Filters

Horizontal Sobel (G_x):

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Vertical Sobel (G_y):

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Usage: Edge detection, gradient magnitude computation.

3.3.2 Prewitt Filters

Horizontal:

$$P_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Vertical:

$$P_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Usage: Simple edge detection; less smoothing than Sobel.

3.4 Nonlinear Filters

3.4.1 Median Filter

Replaces each pixel with the median of the neighborhood.

$$g[m, n] = \text{median}\{f[i, j] \mid (i, j) \in \mathcal{N}_{m, n}\}$$

Usage: Excellent for salt-and-pepper noise removal; preserves edges.

3.4.2 Max Filter

$$g[m, n] = \max\{f[i, j] \mid (i, j) \in \mathcal{N}_{m, n}\}$$

Usage: Extracts bright regions; used in morphological dilation as well.

3.4.3 Min Filter

$$g[m, n] = \min\{f[i, j] \mid (i, j) \in \mathcal{N}_{m, n}\}$$

Usage: Extracts dark regions; related to morphological erosion.

3.4.4 Midpoint Filter

$$g[m, n] = \frac{1}{2}(\max + \min)$$

Usage: Useful for uniform noise reduction.

3.5 Bandpass and Bandreject Filters (Spatial Approximations)

3.5.1 High-Pass Approximation

$$H_{HP} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Usage: Emphasizes edges and fine detail.

3.5.2 Low-Pass Approximation

Same as averaging or Gaussian.

3.5.3 Bandpass

$$H_{BP} = H_{HP} * H_{LP}$$

Usage: Texture extraction, noise isolation.

3.6 Directional Filters

3.6.1 Horizontal Line Detector

$$H = \begin{bmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{bmatrix}$$

3.6.2 Vertical Line Detector

$$H = \begin{bmatrix} -1 & 2 & -1 \\ -1 & 2 & -1 \\ -1 & 2 & -1 \end{bmatrix}$$

Usage: Extraction of oriented features (roads, edges, lines).

3.7 Summary

- Smoothing filters reduce noise but blur edges.
- Sharpening filters enhance edges and fine detail.
- Gradient filters detect edges using first derivatives.
- Nonlinear filters (median, min/max) are essential for salt-and-pepper noise.
- Directional filters detect oriented structures.

3.8 Summary

- Filtering modifies an image using either linear or nonlinear operations.
- Smoothing filters reduce noise; sharpening filters enhance edges.
- Median filters are robust against impulse noise.
- Filters can be implemented in spatial or frequency domains depending on the application.

4 Fourier Transform

4.1 Theory

The Fourier Transform is one of the most powerful tools in image processing. It decomposes an image into its sinusoidal frequency components, representing it in the frequency domain.

High frequencies correspond to:

- edges,
- fine details,
- rapid intensity changes.

Low frequencies correspond to:

- smooth variations,
- illumination gradients,
- background components.

Fourier methods are essential for:

- filtering (low-pass, high-pass, bandpass),
- image restoration,

- compression,
- spectral analysis,
- PSF estimation.

4.2 Mathematical Formulation

4.2.1 Continuous 2D Fourier Transform

For a continuous image $f(x, y)$:

$$F(u, v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) e^{-j2\pi(ux+vy)} dx dy \quad (4)$$

Inverse transform:

$$f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v) e^{j2\pi(ux+vy)} du dv \quad (5)$$

4.2.2 Discrete 2D Fourier Transform (DFT)

Digital images use the discrete form:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)} \quad (6)$$

Inverse DFT:

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)} \quad (7)$$

Magnitude and Phase:

$$|F(u, v)| = \sqrt{\Re^2 + \Im^2}, \quad \phi(u, v) = \tan^{-1} \left(\frac{\Im}{\Re} \right)$$

Key fact: Image structure comes primarily from the **phase**.

4.3 Key Properties of the Fourier Transform

4.3.1 Linearity

$$\mathcal{F}\{af + bg\} = aF + bG$$

4.3.2 Shift Property

A spatial shift introduces a phase shift:

$$f(x - x_0, y - y_0) \xleftrightarrow{\mathcal{F}} e^{-j2\pi(ux_0/M + vy_0/N)} F(u, v)$$

4.3.3 Convolution Theorem

$$f * h \xleftrightarrow{\mathcal{F}} FH$$

Most important result in image filtering.

4.3.4 Correlation Theorem

$$f \circ h \xleftrightarrow{\mathcal{F}} FH^*$$

4.3.5 Separability

If

$$H(u, v) = H_1(u)H_2(v),$$

then filtering can be performed using 1D transforms (computationally efficient).

4.3.6 Symmetry for Real Images

For real-valued images:

$$F(-u, -v) = F^*(u, v)$$

4.3.7 Centering the Fourier Transform

To move the DC component to the center:

$$f_c(x, y) = f(x, y)(-1)^{x+y}$$

4.4 Interpretation in Image Processing

- Low frequencies = illumination + smooth background.
- High frequencies = edges, fine textures.
- Filtering in frequency domain is multiplication:

$$G(u, v) = H(u, v)F(u, v)$$

- Noise removal, deblurring, and PSF inversion rely heavily on Fourier analysis.
- Compression (JPEG) uses frequency decomposition (via DCT).

4.5 Example Problem 1: Fourier Transform of a Separable Image

Problem

Let

$$f(x, y) = a(x)b(y)$$

where $a(x) = [1, 2]$ and $b(y) = [3, 1]$ (1D signals). Compute the 2D DFT $F(u, v)$.

Solution

Since $f(x, y)$ is separable:

$$F(u, v) = A(u)B(v)$$

Compute 1D DFT of $a(x)$:

$$A(0) = 1 + 2 = 3$$

$$A(1) = 1 - 2 = -1$$

Compute 1D DFT of $b(y)$:

$$B(0) = 3 + 1 = 4$$

$$B(1) = 3 - 1 = 2$$

Then the outer product:

$$F = \begin{bmatrix} A(0)B(0) & A(0)B(1) \\ A(1)B(0) & A(1)B(1) \end{bmatrix} = \begin{bmatrix} 3 \cdot 4 & 3 \cdot 2 \\ -1 \cdot 4 & -1 \cdot 2 \end{bmatrix}$$

Thus:

$$F = \begin{bmatrix} 12 & 6 \\ -4 & -2 \end{bmatrix}$$

Interpretation

The entire 2D spectrum can be computed from two 1D transforms — illustrating separability.

4.6 Example Problem 2: Effect of Image Shift on Fourier Transform

Problem

If

$$g(x, y) = f(x - 2, y - 3),$$

derive the Fourier transform $G(u, v)$ in terms of $F(u, v)$.

Solution

Using the shift property:

$$g(x, y) \xleftrightarrow{\mathcal{F}} e^{-j2\pi(\frac{2u}{M} + \frac{3v}{N})} F(u, v)$$

Thus:

$$G(u, v) = F(u, v)e^{-j2\pi(\frac{2u}{M} + \frac{3v}{N})}$$

Interpretation

A spatial shift does **not** change magnitude, only phase — demonstrating why phase contains structural information.

4.7 Summary

- The Fourier Transform converts images into frequency components.
- High and low frequencies correspond to edges and smooth regions, respectively.
- The convolution theorem enables efficient frequency-domain filtering.
- Shifts in space produce phase changes in the frequency domain.
- Fourier analysis is foundational for restoration, enhancement, compression, and PSF modeling.

5 Inverse Fourier Transform

5.1 Theory

The Inverse Fourier Transform (IFT) reconstructs an image from its frequency domain representation. While the Fourier Transform decomposes an image into sinusoidal components, the inverse transform recombines them into the original spatial-domain image.

In image processing, the IFT is essential for:

- reconstructing filtered images (after applying $G(u, v) = H(u, v)F(u, v)$),
- image restoration and deblurring,
- reconstruction in MRI, CT, and other spectral imaging applications,
- transitioning between spatial and frequency domain algorithms,
- verifying linear shift-invariant (LSI) system models.

The forward and inverse transforms form a dual pair, allowing lossless switching between domains (for ideal sampling conditions).

5.2 Mathematical Formulation

5.2.1 Continuous 2D Inverse Fourier Transform

Given the Fourier Transform:

$$F(u, v) = \iint f(x, y) e^{-j2\pi(ux+vy)} dx dy$$

The inverse is:

$$f(x, y) = \iint F(u, v) e^{j2\pi(ux+vy)} du dv$$

The positive sign in the exponential restores the spatial-domain signal.

5.2.2 Discrete 2D Inverse Fourier Transform (IDFT)

For digital images, the inverse is:

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)} \quad (8)$$

Key Points:

- The normalization factor $\frac{1}{MN}$ ensures correct scaling.
- When $F(u, v)$ is complex, the reconstructed $f(x, y)$ is real if the input is real.
- Image-processing software (MATLAB, NumPy, OpenCV) uses the same definition.

5.3 Properties of the Inverse Fourier Transform

5.3.1 1. Duality

If

$$\mathcal{F}\{f(x, y)\} = F(u, v),$$

then

$$\mathcal{F}^{-1}\{F(x, y)\} = f(-u, -v)$$

5.3.2 2. Uniqueness

If $f(x, y)$ is real and finite, then $F(u, v)$ uniquely determines the image.

5.3.3 3. Hermitian Symmetry

For real images $f(x, y)$:

$$F(-u, -v) = F^*(u, v)$$

This ensures IFT yields a real-valued image.

5.3.4 4. Energy Conservation (Parseval's Theorem)

$$\sum_{x,y} |f(x,y)|^2 = \frac{1}{MN} \sum_{u,v} |F(u,v)|^2$$

Useful for comparing filtered vs. noisy images.

5.3.5 5. Circular Convolution

DFT and IDFT obey circular convolution:

$$f * h \leftrightarrow FH$$

Padding is needed to avoid wraparound artifacts.

5.4 Interpretation in Image Processing

- Every frequency-domain filter must be returned to the spatial domain using the IFT.
- Highpass, lowpass, bandpass, and sharpening filters are implemented as:

$$g(x,y) = \mathcal{F}^{-1}\{H(u,v)F(u,v)\}$$

- Image restoration (inverse filtering, Wiener filtering) relies entirely on this process.
- After applying FFT-based convolution, the IFT retrieves the processed image:

$$g = \text{IFFT}(\text{FFT}(f) \cdot \text{FFT}(h))$$

5.5 Example Problem 1: Recovering Spatial Image from Simple Spectrum

Problem

Given the 2D DFT:

$$F = \begin{bmatrix} 12 & 0 \\ 0 & -4 \end{bmatrix}$$

for a 2×2 image, compute the inverse DFT to obtain $f(x,y)$.

Solution

Using the IDFT:

$$f(x,y) = \frac{1}{4} \sum_{u=0}^1 \sum_{v=0}^1 F(u,v) e^{j2\pi(\frac{ux}{2} + \frac{vy}{2})}$$

Compute all four values:

$$f(0,0) = \frac{1}{4}(12 + 0 + 0 - 4) = 2$$

$$f(0,1) = \frac{1}{4}(12 - 0 + 0 + 4) = 4$$

$$f(1,0) = \frac{1}{4}(12 + 0 - 0 + 4) = 4$$

$$f(1,1) = \frac{1}{4}(12 - 0 - 0 - 4) = 2$$

Thus:

$$f = \begin{bmatrix} 2 & 4 \\ 4 & 2 \end{bmatrix}$$

Interpretation

Low frequencies dominate $\rightarrow$ smooth spatial pattern with symmetric values.

5.6 Example Problem 2: IFT After Filtering

Problem

A 2×2 image has DFT:

$$F = \begin{bmatrix} 20 & -4 \\ -4 & 0 \end{bmatrix}$$

A lowpass filter keeps only $F(0,0)$ and sets others to zero. Compute the spatial-domain output.

Solution

Filtered spectrum:

$$G = \begin{bmatrix} 20 & 0 \\ 0 & 0 \end{bmatrix}$$

Apply IDFT:

$$g(x,y) = \frac{1}{4}(20)$$

Thus all pixels become:

$$g = 5 \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

Interpretation

Only the DC component remains $\rightarrow$ completely smoothed (constant) image.

5.7 Summary

- The inverse Fourier Transform reconstructs an image from its spectral representation.
- Filtering in the frequency domain always requires the IFT to obtain spatial results.
- Properties such as Hermitian symmetry ensure real-valued outputs.
- The IFT is essential for restoration, filtering, and frequency-based compression.

6 Quantization

6.1 Theory

Quantization is the process of mapping a continuous or large-range set of pixel intensity values into a finite set of discrete levels. In digital image processing, an image must be quantized in intensity to convert it from an analog signal into a digital representation.

An image with bit depth b has:

$$L = 2^b \text{ quantization levels}$$

Quantization introduces:

- **rounding error (quantization error)**
- **loss of fine detail**
- **contouring artifacts** (visible steps in smooth gradients)

Sources of quantization:

- Analog-to-digital conversion (ADC)
- Digital compression algorithms
- Display systems (e.g., 8-bit grayscale monitors)

6.2 Mathematical Formulation

Let the continuous intensity range be:

$$[I_{\min}, I_{\max}]$$

For uniform quantization with L levels, the quantization step is:

$$\Delta = \frac{I_{\max} - I_{\min}}{L - 1}$$

A pixel value r is mapped to:

$$Q(r) = \Delta \cdot \text{round} \left(\frac{r - I_{\min}}{\Delta} \right) + I_{\min}$$

6.2.1 Quantization Error

The quantization error is:

$$e_q = r - Q(r)$$

For uniform quantizers, it is bounded by:

$$-\frac{\Delta}{2} \leq e_q \leq \frac{\Delta}{2}$$

Assuming a uniform distribution of error:

$$\sigma_e^2 = \frac{\Delta^2}{12}$$

6.2.2 Signal-to-Noise Ratio (SNR)

For an image with variance σ_f^2 , the quantization SNR is:

$$\text{SNR}_q = 10 \log_{10} \left(\frac{\sigma_f^2}{\sigma_e^2} \right)$$

As b increases, Δ decreases $\rightarrow$ better image quality.

6.3 Types of Quantizers

6.3.1 Uniform Quantization

$$Q(r) = \Delta \cdot \text{round} \left(\frac{r}{\Delta} \right)$$

Used in:

- Standard grayscale imaging (8-bit, 12-bit, 16-bit)
- JPEG and DCT-based quantizers (uniform in transformed domain)

6.3.2 Non-Uniform Quantization

More levels are allocated to frequently occurring intensity values.

Optimal quantizers minimize mean square error:

$$Q^* = \arg \min_Q E[(f - Q(f))^2]$$

Examples:

- Lloyd-Max quantizer
- Mu-law / A-law quantization (for compressed sensing or HDR imaging)

6.4 Effects of Quantization in Imaging

6.4.1 False Contouring

Occurs when L is too small. Gradients appear as visible step edges (e.g., sky in 4-bit images).

6.4.2 Loss of Detail

Small variations within a region are merged into the same quantization bin.

6.4.3 Quantization Noise in Smooth Areas

Low-variance regions show more visible quantization noise.

Mitigation Strategies:

- Increase bit depth
- Apply dithering
- Use nonlinear quantization

6.5 Example Problem 1: Quantization Error and SNR

Problem

An image has pixel intensities in $[0, 255]$. It is quantized to 4 bits. Compute:

1. Quantization step Δ
2. Maximum quantization error
3. Quantization noise variance
4. SNR if the image has variance $\sigma_f^2 = 1200$

Solution

Number of levels:

$$L = 2^4 = 16$$

(1) Quantization step:

$$\Delta = \frac{255 - 0}{16 - 1} = \frac{255}{15} = 17$$

(2) Max error:

$$|e_q| \leq \frac{\Delta}{2} = 8.5$$

(3) Noise variance:

$$\sigma_e^2 = \frac{\Delta^2}{12} = \frac{17^2}{12} = \frac{289}{12} = 24.08$$

(4) SNR:

$$\text{SNR} = 10 \log_{10} \left(\frac{1200}{24.08} \right) = 10 \log_{10}(49.8) = 10 \cdot 1.697 = 16.97 \text{ dB}$$

Interpretation

A 4-bit image shows significant quantization noise—SNR of only 17 dB is poor for imaging.

6.6 Example Problem 2: False Contouring Analysis

Problem

A smooth gradient image is quantized from 256 gray levels to 8 levels. Determine:

1. The quantization step size.
2. Whether false contouring will be visually noticeable.

Solution

$$L = 8$$
$$\Delta = \frac{255}{7} \approx 36.4$$

This means any gradual change smaller than 36 gray levels maps to the same output.

Human visual system detects changes as low as 2–3 gray levels, so 36-level jumps are extremely noticeable.

Conclusion

Strong false contouring will be visible in sky regions, medical ultrasound scans, or any smoothly varying image.

6.7 Summary

- Quantization reduces continuous intensity values to discrete levels.
- It introduces quantization error, modeled as uniform noise.
- Low bit-depth images show false contouring and noise.
- SNR improves by roughly 6 dB for every additional quantization bit.

7 Histogram Equalization

7.1 Theory

Histogram Equalization (HE) is a spatial domain technique used to improve global image contrast by redistributing the pixel intensities so that the output image has a *uniform* histogram. Unlike simple contrast stretching, HE is automatic and data-driven.

The method comes directly from probability theory: If we apply a transformation based on the cumulative distribution function (CDF),

$$s = T(r)$$

then the transformed random variable s becomes **uniformly distributed** in $[0, 1]$.

Histogram equalization applies this idea to discrete image intensities.

7.2 Mathematical Derivation (Based on Gonzalez and Woods)

Let an image have gray levels:

$$r_k, \quad k = 0, 1, \dots, L - 1$$

and a histogram giving pixel counts n_k .

Define the probability of gray level:

$$p_r(r_k) = \frac{n_k}{MN}$$

where $M \times N$ is the image size.

7.2.1 1. Transformation Function

The histogram equalization transform is:

$$s_k = T(r_k) = (L - 1) \sum_{j=0}^k p_r(r_j)$$

This is simply the CDF scaled to the range $[0, L - 1]$.

- If many pixels are in a low-intensity region, the CDF rises quickly $\rightarrow$ stretches the dark region.
- If few pixels appear in bright regions, the CDF rises slowly $\rightarrow$ compresses bright regions.

Thus contrast is enhanced by redistributing pixel intensities.

7.2.2 2. Why Histogram Equalization Works

For a continuous function, if:

$$s = T(r)$$

where T is strictly increasing and differentiable, and:

$$T(r) = \int_0^r p_r(\omega) d\omega,$$

then it can be shown that:

$$p_s(s) = 1$$

for $s \in [0, 1]$.

Meaning: **the output distribution becomes uniform.**

In discrete digital images, uniformity is approximate—not exact—but the contrast enhancement effect remains very strong.

7.3 Algorithmic Steps (Discrete Implementation)

Given an image with intensity range $[0, L - 1]$:

1. Compute histogram $h(r_k)$

$$h(r_k) = n_k$$

2. Normalize to obtain PMF

$$p_r(r_k) = \frac{n_k}{MN}$$

3. Compute CDF

$$C(k) = \sum_{j=0}^k p_r(r_j)$$

4. Multiply by $(L - 1)$

$$s_k = (L - 1)C(k)$$

5. Apply mapping: Every pixel of value r_k is replaced by $\lfloor s_k \rfloor$.

7.4 Interpretation

Histogram equalization:

- Spreads out frequently occurring intensities
- Increases global contrast
- Enhances details in dark and bright regions simultaneously
- Is fully automatic — no parameters required

However:

- It may over-enhance noise in homogeneous regions.
- It may produce unnatural brightness for images intended for human interpretation (e.g., medical).

7.5 Local Histogram Equalization (Adaptive HE and CLAHE)

Global HE uses the histogram of the entire image $\rightarrow$ fails when illumination is nonuniform.

7.5.1 Local HE

Break image into regions (e.g., 8×8 blocks):

1. Perform HE on each block.
2. Interpolate block boundaries to avoid artifacts.

7.5.2 CLAHE (Contrast Limited AHE)

To prevent noise amplification:

- Clip histogram at a threshold (the "clip limit").
- Redistribute clipped counts uniformly.
- Compute CDF and equalize.

CLAHE is widely used in:

- Medical imaging (X-ray, CT)
- Night-vision enhancement
- Low-light photography

7.6 Example Problem 1: Manual Histogram Equalization

Problem

Consider a 3×3 image with intensity levels 0, 1, 2, 3:

$$f = \begin{bmatrix} 0 & 1 & 1 \\ 2 & 3 & 3 \\ 3 & 2 & 1 \end{bmatrix}$$

Perform histogram equalization.

Solution

1. Histogram

$$h(0) = 1, \quad h(1) = 3, \quad h(2) = 2, \quad h(3) = 3$$

$$MN = 9, \quad L = 4$$

2. PMF

$$p_r = \{1/9, 3/9, 2/9, 3/9\}$$

3. CDF

$$C(0) = 1/9, \quad C(1) = 4/9, \quad C(2) = 6/9, \quad C(3) = 9/9$$

4. Mapping

$$s_k = (L - 1)C(k) = 3C(k)$$

$$s_0 = 0.33 \rightarrow 0$$

$$s_1 = 1.33 \rightarrow 1$$

$$s_2 = 2.00 \rightarrow 2$$

$$s_3 = 3.00 \rightarrow 3$$

Thus the mapping is:

$$0 \rightarrow 0, \quad 1 \rightarrow 1, \quad 2 \rightarrow 2, \quad 3 \rightarrow 3$$

Since the original histogram already had reasonably spread intensities, equalization does not change output much.

Result

$$g = f$$

7.7 Example Problem 2: Histogram Equalization on a Low-Contrast Image

Problem

A small low-contrast image has intensities in $\{50, 52, 55\}$. Its histogram is:

$$h(50) = 5, \quad h(52) = 2, \quad h(55) = 1$$

Perform histogram equalization assuming 8-bit output ($L = 256$).

Solution

Compute PMF:

$$p_r(50) = 5/8, \quad p_r(52) = 2/8, \quad p_r(55) = 1/8$$

CDF:

$$C(50) = 5/8, \quad C(52) = 7/8, \quad C(55) = 1$$

Mapping to 0–255 range:

$$s(50) = 255(5/8) = 159$$

$$s(52) = 255(7/8) = 223$$

$$s(55) = 255(1) = 255$$

Interpretation The intensities $\{50, 52, 55\}$ are now remapped to $\{159, 223, 255\}$:

- Very large contrast expansion
- Highlights previously invisible details
- Corrects for the low dynamic range of the original image

7.8 Summary

- Histogram Equalization uses the CDF as a transformation function.
- It expands frequently occurring intensity ranges $\rightarrow$ increases contrast.
- It is automatic but global; local variants (AHE, CLAHE) improve robustness.
- HE is fundamental for preprocessing in vision, medical imaging, and remote sensing.

8 Degradation and the Point Spread Function (PSF)

8.1 Introduction

Image degradation refers to any process that causes the captured image to differ from the ideal scene. Examples include:

- motion blur,
- atmospheric turbulence,
- out-of-focus blur,

- sensor imperfections and noise.

Mathematically, degradation is modeled as a linear, shift-invariant (LSI) system characterized by a **Point Spread Function (PSF)**.

8.2 Degradation Model

The most general degradation model in the spatial domain is:

$$g(x, y) = h(x, y) * f(x, y) + \eta(x, y) \quad (9)$$

Where:

- $f(x, y)$: original image
- $h(x, y)$: point spread function (PSF)
- $g(x, y)$: observed degraded image
- $\eta(x, y)$: additive noise
- $*$: 2D convolution

Taking the Fourier transform yields:

$$G(u, v) = H(u, v)F(u, v) + N(u, v) \quad (10)$$

Where $H(u, v)$ is the **Optical Transfer Function (OTF)**, i.e., the Fourier transform of the PSF.

8.3 Point Spread Function (PSF)

The PSF describes the response of an imaging system to a point source of light. In ideal imaging, the PSF is a delta function:

$$h(x, y) = \delta(x, y)$$

meaning no blurring.

Real-world systems spread light, producing a blur kernel.

8.3.1 Properties of PSF

- Spatial domain representation of blur
- Must be non-negative
- Must integrate to 1 (energy preservation)

$$\iint h(x, y) dx dy = 1$$

- Determines the frequency response:

$$H(u, v) = \mathcal{F}\{h(x, y)\}$$

8.4 Common PSF Models in Imaging

8.4.1 1. Motion Blur PSF (Uniform Linear Motion)

If camera or object moves uniformly during exposure, the PSF is:

$$h(x, y) = \frac{1}{L} \text{sinc-like blur along motion direction}$$

More precisely, for motion of length L at angle θ :

$$h(x, y) = \begin{cases} \frac{1}{L}, & 0 \leq x \cos \theta + y \sin \theta \leq L \\ 0, & \text{otherwise} \end{cases}$$

Its frequency response:

$$H(u, v) = \frac{\sin(\pi(u \cos \theta + v \sin \theta)L)}{\pi(u \cos \theta + v \sin \theta)L}$$

8.4.2 2. Out-of-Focus Blur (Defocus Blur)

A circular aperture gives:

$$h(x, y) = \begin{cases} \frac{1}{\pi R^2}, & x^2 + y^2 \leq R^2 \\ 0, & \text{otherwise} \end{cases}$$

OTF:

$$H(u, v) = \frac{2J_1(2\pi RD)}{2\pi RD}$$

where J_1 is the Bessel function and $D = \sqrt{u^2 + v^2}$.

8.4.3 3. Atmospheric Turbulence Blur

Modeled using:

$$H(u, v) = e^{-k(u^2 + v^2)^{5/6}}$$

Used in satellite and astronomy imaging.

8.4.4 4. Gaussian Blur PSF

For optical scattering:

$$h(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

Fourier transform:

$$H(u, v) = \exp(-2\pi^2\sigma^2(u^2 + v^2))$$

This is widely used due to its simplicity and physical relevance.

8.5 Noise in the Degradation Model

Noise is typically independent of blur. Common noise distributions:

- Gaussian noise (sensor/thermal noise)
- Salt-and-pepper noise (bit errors)
- Poisson noise (photon counting)

Noise makes restoration far more challenging.

8.6 Why PSF Matters in Restoration

All restoration methods require knowledge or estimation of the PSF.

- **Inverse filtering:**

$$\hat{F}(u, v) = \frac{G(u, v)}{H(u, v)}$$

Requires exact PSF.

- **Wiener filtering:**

$$\hat{F}(u, v) = \frac{H^*(u, v)}{|H(u, v)|^2 + S_\eta(u, v)/S_f(u, v)} G(u, v)$$

- **Constrained least squares (CLS).**
- **Blind deconvolution:** Estimate h and f simultaneously.

8.7 Example Problem 1: Motion Blur PSF and OTF

Problem

A camera moves horizontally by 10 pixels during exposure. Assume motion blur PSF:

$$h(x, y) = \begin{cases} \frac{1}{10}, & 0 \leq x \leq 10, y = 0 \\ 0, & \text{otherwise} \end{cases}$$

Find the frequency response $H(u, v)$.

Solution

By definition:

$$H(u, v) = \int_0^{10} \frac{1}{10} e^{-j2\pi ux} dx$$

Compute integral:

$$H(u, v) = \frac{1}{10} \left[\frac{e^{-j2\pi ux}}{-j2\pi u} \right]_0^{10}$$

$$H(u, v) = \frac{1}{10} \cdot \frac{1}{j2\pi u} (1 - e^{-j20\pi u})$$

Using Euler form:

$$H(u, v) = \frac{\sin(10\pi u)}{10\pi u}$$

$$\boxed{H(u, v) = \text{sinc}(10u)}$$

This is the classical motion-blur frequency response.

8.8 Example Problem 2: Gaussian PSF Degradation

Problem

An imaging system has Gaussian PSF:

$$h(x, y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/(2\sigma^2)}$$

Find the OTF and determine at what frequency the magnitude drops to 0.5.

Let $\sigma = 2$.

Solution

The OTF of a Gaussian is:

$$H(u, v) = e^{-2\pi^2\sigma^2(u^2+v^2)}$$

Set:

$$|H| = 0.5$$

$$0.5 = e^{-2\pi^2(4)(u^2+v^2)}$$

Take logs:

$$\ln 0.5 = -8\pi^2(u^2 + v^2)$$

$$u^2 + v^2 = \frac{\ln 2}{8\pi^2}$$

Numerically:

$$u^2 + v^2 \approx \frac{0.693}{78.96} = 0.00878$$

Thus the radius in frequency domain at half-power is:

$$D = \sqrt{0.00878} \approx 0.094$$

Meaning the system strongly suppresses frequencies above 0.1 cycles/pixel.

8.9 Summary

- The PSF models blur in an imaging system.
- Degradation model:

$$g = h * f + \eta$$

- PSF's Fourier transform is the OTF, essential in restoration.
- Common PSFs model motion, defocus, atmospheric, and Gaussian blur.
- Restoration methods (inverse, Wiener, CLS) depend on accurate PSF knowledge.

9 Discrete Cosine Transform (DCT)

9.1 Introduction

The Discrete Cosine Transform (DCT) is one of the most important transforms in image processing, especially in compression standards such as JPEG, MPEG, and H.26x. It provides excellent **energy compaction**, meaning most of the image energy is packed into a few low-frequency coefficients.

The DCT is closely related to the Fourier Transform, but uses only cosine basis functions, making it:

- real-valued (no imaginary components),
- even-symmetric,
- better suited for natural images,
- less sensitive to boundary discontinuities.

The most commonly used version is the **2D DCT-II**, which is the mathematical core of JPEG.

9.2 1D DCT Definition

For a 1D signal $f(x)$ of length N :

$$F(u) = \alpha(u) \sum_{x=0}^{N-1} f(x) \cos \left[\frac{\pi(2x+1)u}{2N} \right] \quad (11)$$

Where:

$$\alpha(u) = \begin{cases} \sqrt{\frac{1}{N}}, & u = 0 \\ \sqrt{\frac{2}{N}}, & u > 0 \end{cases}$$

The inverse DCT (IDCT) is:

$$f(x) = \sum_{u=0}^{N-1} \alpha(u) F(u) \cos \left[\frac{\pi(2x+1)u}{2N} \right] \quad (12)$$

9.3 2D DCT (Most Relevant for Images)

For an $N \times N$ block (JPEG uses $N = 8$):

$$F(u, v) = \alpha(u)\alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos \left[\frac{\pi(2x+1)u}{2N} \right] \cos \left[\frac{\pi(2y+1)v}{2N} \right] \quad (13)$$

The inverse transform is:

$$f(x, y) = \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} \alpha(u)\alpha(v) F(u, v) \cos \left[\frac{\pi(2x+1)u}{2N} \right] \cos \left[\frac{\pi(2y+1)v}{2N} \right] \quad (14)$$

9.4 Interpretation of DCT Coefficients

In the DCT domain:

- $F(0, 0)$ = DC coefficient (average intensity of the block).
- Low-frequency coefficients $\rightarrow$ smooth variations.
- High-frequency coefficients $\rightarrow$ edges, noise, details.

Natural images tend to concentrate most energy in low-frequency components. Thus high-frequency components can be quantized more aggressively $\rightarrow$ basis of JPEG compression.

9.5 Why the DCT Compacts Energy Well

Consider an $N \times N$ image block:

- Smooth regions $\rightarrow$ cosine basis aligns well $\rightarrow$ large values in DC and few low-frequency coefficients.
- Sharp edges $\rightarrow$ energy spreads, but still more compact than Fourier.
- Cosines reduce boundary discontinuity artifacts because they assume even symmetry.

9.6 DCT in JPEG Compression

JPEG applies DCT as follows:

1. Convert RGB to YCbCr.
2. Divide image into 8×8 blocks.
3. Apply 2D DCT to each block.

4. Quantize each coefficient using quantization matrix $Q(u, v)$:

$$\hat{F}(u, v) = \text{round} \left(\frac{F(u, v)}{Q(u, v)} \right)$$

5. Zig-zag scan (run-length encoding).
6. Entropy coding (Huffman or arithmetic coding).

The DCT is thus the main step determining compression quality.

9.7 Example Problem 1: Compute DCT of a Simple 1D Signal

Problem

Given a 1D signal of length 4:

$$f(x) = [10, 10, 10, 10]$$

Compute the DCT coefficients.

Solution

Since the signal is constant:

$$F(0) = \sqrt{\frac{1}{4}}(10 + 10 + 10 + 10) = \frac{1}{2}(40) = 20$$

For $u > 0$, cosine terms integrate to zero due to orthogonality:

$$F(1) = F(2) = F(3) = 0$$

Thus:

$$F = [20, 0, 0, 0]$$

Interpretation

All energy is in the DC component $\rightarrow$ perfect energy compaction.

9.8 Example Problem 2: 2D DCT of an 2×2 Block

Problem

Compute the 2D DCT of:

$$f = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

Use $N = 2$ and:

$$\alpha(0) = \frac{1}{\sqrt{2}}, \quad \alpha(1) = 1$$

Solution

Compute $F(0,0)$ (DC term):

$$F(0,0) = \alpha(0)\alpha(0) \sum f(x,y) = \frac{1}{2}(4) = 2$$

Compute $F(0,1)$: Cosine terms for $v = 1$ alternate:

$$F(0,1) = \alpha(0)\alpha(1)[1 - 1 + 1 - 1] = 0$$

Compute $F(1,0)$:

$$F(1,0) = \alpha(1)\alpha(0)[1 + 1 - 1 - 1] = 0$$

Compute $F(1,1)$:

$$F(1,1) = 1 \cdot 1[1 - 1 - 1 + 1] = 0$$

Thus:

$$F = \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}$$

Interpretation

Again, all energy is in the DC coefficient because the block is constant.

9.9 Summary

- The DCT uses cosine basis functions to transform an image block into frequency components.
- Excellent for energy compaction $\rightarrow$ crucial for JPEG.
- DC component represents average intensity; higher frequencies represent details.
- High-frequency coefficients can be quantized more aggressively with minimal perceptual loss.

10 Text-Based Data Compression

10.1 Introduction

Data compression refers to representing information using fewer bits than the original representation. Compression is essential in storage, transmission, and multimedia systems (e.g., JPEG, PNG, MPEG).

Even though this topic arises in the context of text compression, the principles apply directly to *image compression*, where pixel intensities or transformed coefficients are treated as symbols.

10.2 Types of Redundancy

Compression works by removing redundancy. According to Gonzalez and Woods and classical information theory, there are three primary types:

1. **Coding redundancy** Some symbols use more bits than necessary. Example: fixed-length ASCII (8 bits) for 26 letters.
2. **Spatial (interpixel) redundancy** Pixels in images are correlated $\rightarrow$ predictable. Used by predictive coding, transform coding, LZ-family algorithms.
3. **Psychovisual redundancy** The human visual system (HVS) is less sensitive to high frequencies. Used in JPEG quantization.

Lossless compression removes (1) and (2). Lossy compression additionally removes (3).

10.3 Information Theory Basics

10.3.1 Entropy

Let symbols have probabilities p_i . The entropy of the source is:

$$H = - \sum_{i=1}^N p_i \log_2 p_i \quad (15)$$

Properties:

- H is the lower bound (in bits/symbol) of any lossless code.
- More skewed distributions $\rightarrow$ lower entropy.

10.3.2 Average Codeword Length

If a code assigns lengths l_i to symbol i , the average length is:

$$L = \sum_{i=1}^N p_i l_i \quad (16)$$

An optimal code satisfies:

$$H \leq L < H + 1$$

Huffman and arithmetic coding approach this bound.

10.4 Prefix Codes

A prefix code is a code where no codeword is a prefix of another.

Properties:

- Instantaneously decodable
- Satisfy Kraft-McMillan inequality:

$$\sum_{i=1}^N 2^{-l_i} \leq 1$$

- Basis for Huffman coding and many entropy codes

10.5 Lossless vs. Lossy Compression

10.5.1 Lossless Compression

- Exact reconstruction
- Used for PNG, GIF, medical imaging, LZ77/LZW, text files

10.5.2 Lossy Compression

- Allows controlled distortion
- Used in JPEG, MPEG
- Removes psychovisual redundancy

10.6 Modeling Data for Compression

There are two broad approaches:

1. **Statistical modeling** — probabilities assigned to symbols Used in Huffman, arithmetic coding.
2. **Dictionary modeling** — recurring patterns replaced with references Used in LZ77, LZ78, LZW, PNG.

Both approaches appear in image compression:

- JPEG uses statistical + transform coding.
- PNG uses LZ77.
- TIFF can use LZW.

10.7 Two-Part Structure of Compression Algorithms

Most compression systems consist of:

1. **A redundancy reduction step** (e.g., transform coding, prediction, dictionary encoding)
2. **An entropy coding step** (Huffman or arithmetic coding)

For example, JPEG pipeline:

DCT $\rightarrow$ Quantization $\rightarrow$ Zig-zag $\rightarrow$ Huffman

10.8 Example Problem 1: Compute Entropy and Minimum Bits Required

Problem

A source emits symbols with probabilities:

$$p = \{0.5, 0.25, 0.125, 0.125\}$$

Compute:

1. Entropy
2. Minimum number of bits per symbol (lower bound)
3. Redundancy if average codeword length is 2 bits

Solution

1. Entropy

$$\begin{aligned} H &= - \sum p_i \log_2 p_i \\ &= -[0.5(-1) + 0.25(-2) + 0.125(-3) + 0.125(-3)] \\ &= 0.5 + 0.5 + 0.375 + 0.375 = 1.75 \text{ bits} \end{aligned}$$

2. Minimum bits:

$$L_{\min} = H = 1.75$$

3. Redundancy:

$$R = L - H = 2 - 1.75 = 0.25 \text{ bits/symbol}$$

Interpretation

Even the best code cannot do better than 1.75 bits/symbol.

10.9 Example Problem 2: Verify Kraft Inequality and Prefix Condition

Problem

Consider the following code:

$$A : 0, \quad B : 10, \quad C : 110, \quad D : 111$$

1. Verify Kraft inequality.
2. Determine if this is a prefix-free code.

Solution

1. Kraft inequality

$$\begin{aligned} K &= 2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} \\ &= 0.5 + 0.25 + 0.125 + 0.125 = 1.0 \end{aligned}$$

Since $K \leq 1$, the code is valid.

2. Prefix condition

Check if any codeword is prefix of another:

- 0 is not a prefix of any word starting with 1.
- 10 is not a prefix of 110 or 111.
- 110 and 111 do not overlap.

Thus, the code is **prefix-free**.

Interpretation

The code is uniquely decodable and instantaneous.

10.10 Summary

- Compression reduces redundancy (coding, interpixel, psychovisual).
- Entropy defines the theoretical lower bound for lossless compression.
- Prefix-free codes enable instant decoding.
- Compression uses modeling (dictionary/statistical) + entropy coding.
- This foundation underlies LZ77, LZ78, LZW, Huffman, arithmetic, and Golomb-Rice coding.

11 LZ77 Compression

11.1 Introduction

LZ77 is a dictionary-based, lossless compression algorithm introduced by Lempel and Ziv in 1977. It forms the foundation of many modern compressors, including:

- **DEFLATE** (used in PNG, ZIP, gzip)
- **LZSS, LZMA, Zstd, LZO**

LZ77 replaces repeated sequences with **pointers** referencing previous occurrences within a sliding window.

11.2 Sliding Window Model

LZ77 uses:

- a **search buffer**: previously encoded text
- a **look-ahead buffer**: text to be encoded next



The encoder outputs triples of the form:

$$(d, \ell, s)$$

Where:

- d = distance backward (offset)
- ℓ = length of match
- s = next unmatched symbol

They mean: “Go back d characters, copy ℓ characters, then add symbol s .”

If no match is found:

$$(0, 0, s)$$

is emitted.

11.3 Matching Procedure

1. For the start of the lookahead buffer, search for the longest match in the search buffer.
2. Compute distance d and length ℓ .
3. Emit triple.
4. Slide window by $\ell + 1$ positions.

11.4 Decoding

Decoding simply reverses the pointer:

If (d, ℓ, s) , then output: $\underbrace{\text{copy previous } \ell \text{ characters from offset } d}_{\text{dictionary copy}}$, then write symbol s .

11.5 Why LZ77 Works

Natural language text and image data (after filtering/transforming) contain repeated patterns. Examples:

- Repeated words, prefixes, suffixes.
- Repeated byte sequences in PNG filtered rows.
- Runs of similar symbols after prediction.

Replacing these with backward references achieves significant compression.

11.6 LZ77 in Image Processing

PNG uses DEFLATE = (LZ77 + Huffman coding). Before LZ77, PNG applies a reversible line predictor to convert pixel values into differences $\rightarrow$ increasing redundancy.

Thus LZ77 is indirectly important for lossless image compression.

11.7 Example Problem 1: Manual LZ77 Encoding

Problem

Encode the string:

BANANA BAN

Assume unlimited window size.

Solution

Start:

B: no match

$(0, 0, B)$

A: no match

$(0, 0, A)$

N: no match

$(0, 0, N)$

A: previous A at position 2 $\rightarrow$ match length 1

$$d = 3, \ell = 1, s = N$$

So:

$$(3, 1, N)$$

Next sequence: "ANA BAN"

Match "ANA" with earlier "ANA" starting at position 2:

$$d = 4, \ell = 3, s = \text{space}$$

Output:

$$(4, 3, \text{space})$$

Remaining: "BAN"

We saw "BAN" at the beginning.

$$d = 9, \ell = 3, s = \text{EOF}$$

Result

$$(0, 0, B), (0, 0, A), (0, 0, N), (3, 1, N), (4, 3, \text{space}), (9, 3, \text{EOF})$$

Interpretation

LZ77 replaces repeated substrings with compact references.

11.8 Example Problem 2: LZ77 Decoding Problem

Decode the LZ77 output:

$$(0, 0, h), (0, 0, e), (2, 1, l), (3, 2, o)$$

Solution

Step-by-step:

1. $(0, 0, h) \rightarrow$ output "h"
2. $(0, 0, e) \rightarrow$ "he"
3. $(2, 1, l)$ Look back 2 characters: "he" Copy 1 $\rightarrow$ "h" Output:

"hel"

4. $(3, 2, o)$ Look back 3 characters: substring "hel" Copy 2: "he" Append "o":

hello

So final output:

hello

11.9 Summary

- LZ77 uses a sliding window to replace repeated sequences with backward references.
- Output triples: (d, ℓ, s)
- Basis of PNG, ZIP, gzip.
- Very effective after predictive filtering (as done in image compression).
- Decoding is extremely simple: copy from history and append next symbol.

12 LZ78 Compression

12.1 Introduction

LZ78 (Lempel–Ziv, 1978) is a lossless dictionary-based compression algorithm. Unlike LZ77, which uses a sliding window, LZ78 builds an explicit dictionary of previously seen *phrases*. Each new phrase is assigned an index and added to the dictionary.

LZ78 is fundamental in compression theory and is the basis of:

- LZW (used in GIF and some TIFF variants)
- Many adaptive dictionary compressors

12.2 Encoding Mechanism

LZ78 builds a dictionary of the form:

1 : phrase₁, 2 : phrase₂, ...

Encoding outputs **pairs**:

(i, c)

Where:

- i = dictionary index of the longest matching phrase

- c = next symbol after that phrase

Meaning:

$$\text{dictionary}[i] + c$$

forms the next new phrase.

12.2.1 Encoding Steps

Given input string S :

1. Initialize dictionary with index 0 representing the empty string.
2. Scan input for the longest phrase present in dictionary.
3. Let match index be i , next character be c .
4. Output pair: (i, c) .
5. Insert new phrase $\text{dictionary}[i] + c$ into dictionary.
6. Move forward by the length of the new phrase.

12.3 Decoding

Decoding uses the same dictionary-building process, but reconstructs phrases as follows:

Given pair (i, c) :

$$\text{phrase} = \text{dictionary}[i] + c$$

Append this phrase to the output, then add it to the dictionary.

12.4 Comparison: LZ77 vs. LZ78

LZ77	LZ78
Sliding window dictionary	Explicit, growing dictionary
Dictionary entries overlap	Dictionary entries are stored explicitly
Output: (distance, length, next)	Output: (index, next)
Used by PNG (DEFLATE)	Basis for LZW (GIF, TIFF)
Better for repeated local patterns	Better for repetitive global patterns

12.5 Example Problem 1: LZ78 Encoding

Problem

Encode the string:

ABABABA

using LZ78.

Solution

Start with dictionary:

$0 : \epsilon$

Step 1: Longest match = empty string, next char = A Output:

$(0, A)$

Add phrase 1:

$1 : A$

Step 2: Next input starts with B Match = empty, next = B

$(0, B)$

Add phrase 2:

$2 : B$

Step 3: Next input: ABABA Match = A (index 1), next = B

$(1, B)$

Add phrase 3:

$3 : AB$

Step 4: Remaining: ABA Match = AB (index 3), next = A

$(3, A)$

Add phrase 4:

$4 : ABA$

Encoding complete.

Final Output

$(0, A), (0, B), (1, B), (3, A)$

Interpretation

The dictionary stores meaningful repeated phrases:

A, B, AB, ABA

12.6 Example Problem 2: LZ78 Decoding

Problem

Decode the LZ78 sequence:

$(0, A), (0, B), (1, C), (3, B)$

Solution

Initialize:

$0 : \epsilon$

1. $(0, A)$ Phrase = $\epsilon + A = A$ Output: A Add to dictionary:

$1 : A$

2. $(0, B)$ Phrase = $\epsilon + B = B$ Output: B Add:

$2 : B$

3. $(1, C)$ Dictionary[1] = A Phrase = AC Output: AC Add:

$3 : AC$

4. $(3, B)$ Dictionary[3] = AC Phrase = ACB Output: ACB Add:

$4 : ACB$

Final Output

Concatenating:

$ABACACB$

$ABACACB$

12.7 Summary

- LZ78 uses explicit dictionary entries indexed by integers.
- Output consists of (index, next symbol) pairs.
- Dictionary grows as new phrases are discovered.
- More efficient than LZ77 when repeated phrases appear across long distances.
- Basis of the widely used LZW compression algorithm.

13 LZW Compression

13.1 Introduction

LZW (Lempel–Ziv–Welch, 1984) is an improved variant of LZ78. It is a **lossless**, dictionary-based compression algorithm widely used in:

- GIF images

- TIFF formats
- UNIX compress utility
- PDF streams (optional)

LZW modifies LZ78 so that the encoder outputs **only index codes**, not (index, symbol) pairs. It relies on the fact that the decoder can reconstruct the same dictionary dynamically.

13.2 Key Idea

LZW initializes the dictionary with all possible single-character symbols. For 8-bit data (e.g., grayscale or GIF-style indexed color):

Dictionary initially contains entries 0–255

As encoding proceeds, longer strings are added.

13.3 Encoding Algorithm

Given input string S :

1. Initialize dictionary with all single characters (256 entries).
2. Set w = first character of input.
3. Read next character k .
4. If wk exists in dictionary, set $w \leftarrow wk$.
5. Else:
 - Output code for w
 - Add wk to dictionary
 - Set $w = k$
6. Repeat until end of input, then output code for last w .

13.4 Decoding Algorithm

Given code stream:

1. Initialize dictionary with 256 single-character entries.
2. Read first code $\rightarrow$ output its character $\rightarrow$ set w to that character.
3. For each subsequent code k :
 - If k is in dictionary: entry = dictionary[k]

- Else (edge case): $entry = w + w[0]$
- Output entry
- Add $w + entry[0]$ to dictionary
- Set $w = entry$

13.5 Why LZW Works Well

- Single-character dictionary initialization ensures fast start.
- Long repeated patterns (common in text and image scanlines) are encoded as single codes.
- Dictionary entries grow to reflect statistical structure of data.
- Efficient for images with repeated colors, lines, patterns (e.g., cartoons, diagrams).

13.6 LZW in Image Processing (GIF/TIFF)

GIF operates on indexed images (0–255 color indices). Process:

1. Each pixel value is a symbol in $[0, 255]$.
2. LZW compresses the pixel stream.
3. No loss of image fidelity.

TIFF offers LZW as an optional lossless compression mode.

13.7 Example Problem 1: LZW Encoding

Problem

Encode the string:

ABABABA\$

Assume ASCII values 65=A, 66=B, 36=\$.

Solution

Initial dictionary: 0–255.

Start:

w = A Next symbol: B "A B" not in dictionary → output A=65 Add "AB" at index 256 Set $w = B$

w = B Next: A "BA" not in dictionary → output 66 Add "BA" at 257 $w = A$

w = A Next: B "AB" is in dictionary (256) So set $w = "AB"$

Next: A "ABA" not in dictionary $\rightarrow$ output code for "AB" = 256 Add "ABA" at index 258 Set $w = A$

Next: \$ "A\$" not in dictionary $\rightarrow$ output A=65 Add "A\$" at 259 Set $w = \$$

End of input $\rightarrow$ output \$ = 36

Final Encoded Output

65, 66, 256, 65, 36

Interpretation

Repeated patterns "AB" and "ABA" were detected automatically and encoded as single dictionary references.

13.8 Example Problem 2: LZW Decoding

Problem

Decode the LZW stream:

65, 66, 256, 258, 36

Assume same dictionary initialization (ASCII).

Solution

Initial dictionary: entries 0–255.

Step 1: Code 65 $\rightarrow$ "A" Output: A Set $w = "A"$

Step 2: Code 66 $\rightarrow$ "B" Output: B Add "AB" ($w + B[0]$) at 256 $w = "B"$

Step 3: Code 256 $\rightarrow$ dictionary[256] = "AB" Output: AB Add "BA" at 257 ($w + A$) $w = "AB"$

Step 4: Code 258 $\rightarrow$ not yet defined. Special case: entry = $w + w[0] = "AB" + "A" = "ABA"$ Output: ABA Add "AB" + "A" = "ABA" at 258 $w = "ABA"$

Step 5: Code 36 $\rightarrow$ "\$" Output: \$ Add "ABA" + "\$" at 259

Final Output

Concatenating:

ABABABA\$

13.9 Summary

- LZW initializes dictionary with all symbols, unlike LZ78.
- Output is a sequence of indices only.

- Dictionary is built identically during encoding and decoding.
- Widely used in GIF and TIFF image formats.
- Efficient for repeated raster patterns in images.

14 Encoding Theory

14.1 Introduction

Encoding is the foundation of all lossless compression systems. Given a set of symbols $\mathcal{A} = \{a_1, a_2, \dots, a_n\}$ with associated probabilities

$$P = \{p_1, p_2, \dots, p_n\},$$

the goal is to assign binary codewords

$$C = \{c_1, c_2, \dots, c_n\}$$

such that the expected number of bits per symbol is minimized.

$$L = \sum_{i=1}^n p_i \ell_i$$

where ℓ_i is the length of codeword c_i .

14.2 Uniquely Decodable and Prefix Codes

A code is **uniquely decodable** if every encoded bitstream has exactly one valid decoding.

A more restrictive class, **prefix codes**, ensures:

No codeword is a prefix of any other codeword.

Prefix codes (e.g., Huffman codes) allow instantaneous decoding:

- No look-ahead required
- No ambiguity
- Decoding can proceed bit by bit

Every prefix code corresponds to a binary tree:

- Leaves = symbols
- Path to leaf = codeword
- Left child = 0, right child = 1 (or vice versa)

14.3 Kraft–McMillan Inequality

For any prefix code with codeword lengths $\ell_1, \ell_2, \dots, \ell_n$:

$$\sum_{i=1}^n 2^{-\ell_i} \leq 1$$

Conversely, if a set of lengths satisfies Kraft’s inequality, a prefix code exists.

Proof Sketch: In a full binary tree, a node at depth ℓ occupies a fraction $2^{-\ell}$ of the tree capacity. All codewords must fit into the tree without overlap $\rightarrow$ total “capacity” used must not exceed 1.

This inequality is the basis for validating or designing prefix codes.

14.4 Optimality of Prefix Codes

Shannon’s source coding theorem states:

$$H(X) \leq L < H(X) + 1$$

where

$$H(X) = - \sum p_i \log_2 p_i$$

is the entropy of the source.

A code is optimal when it minimizes expected code length L . Huffman coding achieves the optimal prefix code.

14.5 Entropy and Average Code Length

$$H(X) = - \sum_{i=1}^n p_i \log_2 p_i$$

The lower bound for any uniquely decodable binary code is:

$$L \geq H(X)$$

If probabilities are powers of $1/2$, Huffman coding achieves:

$$L = H(X)$$

14.6 Instantaneous Codes

Instantaneous codes are exactly the prefix codes.

Properties:

- Decoded without delay
- No ambiguity

- Represented by full binary trees

Example:

$$a = 0, b = 10, c = 110, d = 111$$

No codeword is the prefix of any other $\rightarrow$ prefix-free.

14.7 Canonical Codes

Some compressors (e.g., DEFLATE) use **canonical Huffman codes**, which:

- Assign shorter codes to smaller indices
- Require only code lengths to reconstruct full table
- Provide deterministic compact representation

Canonical Huffman is vital for PNG, ZIP, gzip.

14.8 Example Problem 1: Checking Prefix Validity

Problem

Determine whether the following code is a prefix code:

$$a = 0, \quad b = 01, \quad c = 011, \quad d = 111$$

Solution

Check if any codeword is a prefix of another.

- $a = 0$ is a prefix of all others (01, 011).

Thus this is **not** a prefix code.

Additionally, Kraft inequality check:

$$2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{8} = 1$$

Even though Kraft equality holds, prefix condition fails $\rightarrow$ code is uniquely decodable but not instantaneous.

Conclusion

Not a prefix code.

14.9 Example Problem 2: Designing a Prefix Code From Given Lengths

Problem

Given symbol set $\{A, B, C, D\}$ with desired codeword lengths:

$$\ell_A = 1, \ell_B = 2, \ell_C = 3, \ell_D = 3$$

(a) Check if a prefix code is possible. (b) Construct a valid prefix code.

Solution

(a) Kraft Check

$$2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{8} = 1$$

Kraft inequality satisfied $\rightarrow$ prefix code exists.

(b) Constructing Code

Assign shortest codes to earliest leaves:

$$A = 0$$

Remaining tree capacity starts with prefixes beginning with “1”.

$$B = 10$$

Next, two leaves at depth 3:

$$C = 110, D = 111$$

Final Prefix Code

$A = 0, B = 10, C = 110, D = 111$

14.10 Summary

- Encoding assigns binary codewords to symbols to minimize expected length.
- Prefix codes ensure instantaneous decoding.
- Kraft inequality validates prefix code feasibility.
- Shannon entropy gives the lower bound on average code length.
- Canonical codes allow compact representation and efficient decoding.

15 Huffman Coding

15.1 Introduction

Huffman Coding (David Huffman, 1952) is a **lossless**, **optimal** prefix-code algorithm for symbols with known probabilities. It generates a binary tree where:

- high-probability symbols receive short codewords,
- low-probability symbols receive long codewords.

Huffman coding minimizes the expected code length:

$$L = \sum_i p_i \ell_i$$

subject to the prefix condition.

It is widely used in:

- JPEG (as entropy coding after DCT quantization)
- PNG/ZIP/gzip (DEFLATE uses *canonical* Huffman)
- TIFF

15.2 Constructing the Huffman Code

Given symbols $a_1, a_2, \dots, a_n$ with probabilities $p_1, p_2, \dots, p_n$:

1. Create a leaf node for each symbol with weight p_i .
2. Insert all nodes into a min-priority queue.
3. Repeat until a single node remains:
 - Remove two nodes with smallest weights.
 - Create new node with weight = sum of weights.
 - Attach the removed nodes as children.
 - Insert the new node back into the queue.
4. Assign:
left child $\rightarrow 0$, right child $\rightarrow 1$
5. Read codes from root to leaves.

15.3 Properties of Huffman Codes

Optimality Huffman codes minimize expected length among all prefix codes:

$$L_{\text{Huffman}} = \min L$$

Prefix-free No codeword is a prefix of any other.

Full Binary Tree Every internal node has exactly two children.

Length Bounds

$$H(X) \leq L < H(X) + 1$$

Limitations Huffman coding is suboptimal when:

- probabilities change over time (adaptive schemes needed),
- symbol frequencies are uniform (little gain),
- probabilities are extremely skewed (arithmetic coding may outperform).

15.4 Canonical Huffman Coding (Used in PNG/ZIP)

Canonical Huffman does not store tree structure. Instead it stores only *code lengths*.

Decoder reconstructs codes using rules:

1. Sort symbols by increasing length.
2. Shortest codes start from all zeros.
3. Each subsequent code is binary increment of previous.

Canonical coding ensures:

- deterministic assignment,
- compact representation,
- faster decoding via table lookup.

15.5 Example Problem 1: Full Huffman Construction

Problem

Given symbols and probabilities:

$$A : 0.40, \quad B : 0.20, \quad C : 0.20, \quad D : 0.10, \quad E : 0.10$$

Construct the Huffman code and compute average length.

Solution

Step 1: Combine smallest probabilities $D (0.10) + E (0.10) \rightarrow DE (0.20)$

Now we have:

$$A : 0.40, \quad B : 0.20, \quad C : 0.20, \quad DE : 0.20$$

Step 2: Combine next smallest Choose any two of 0.20:

$$B(0.20) + C(0.20) \rightarrow BC(0.40)$$

Step 3: Combine remaining lowest

$$DE(0.20) + (BC)(0.40) \rightarrow DEBC(0.60)$$

Step 4: Combine last two

$$A(0.40) + (DEBC)(0.60) \rightarrow 1.00$$

Assign bits Choose:

Left child = 0 Right child = 1

Resulting codes (one valid assignment):

$$A = 0$$

$$B = 100$$

$$C = 101$$

$$D = 1100$$

$$E = 1101$$

Compute Average Length

$$L = 0.40(1) + 0.20(3) + 0.20(3) + 0.10(4) + 0.10(4)$$

$$L = 0.4 + 0.6 + 0.6 + 0.4 + 0.4 = 2.4$$

Final Answer

$$A = 0, B = 100, C = 101, D = 1100, E = 1101, L = 2.4$$

15.6 Example Problem 2: Canonical Huffman Code Construction

Problem

Symbols have the following codeword lengths:

$$\ell_A = 2, \ell_B = 2, \ell_C = 3, \ell_D = 3, \ell_E = 3$$

Construct the canonical Huffman code.

Solution

Step 1: Sort symbols by length (and alphabetically if tied):

$$A(2), B(2), C(3), D(3), E(3)$$

Step 2: Assign smallest code:

$$A = 00$$

Step 3: Next code is binary increment:

$$B = 01$$

Step 4: Increase length to 3 bits:

Append trailing zeros when moving to next length:

$$C = 100, \quad D = 101, \quad E = 110$$

Final Canonical Codes

$A = 00, B = 01, C = 100, D = 101, E = 110$

15.7 Summary

- Huffman coding produces optimal prefix codes.
- Average code length satisfies $H \leq L < H + 1$.
- Construction uses bottom-up merging of least probable symbols.
- Canonical Huffman codes allow efficient storage and decoding.
- Widely used in image compression (JPEG / PNG).

16 Arithmetic Coding

16.1 Introduction

Arithmetic coding is a form of entropy coding that represents an entire message as a single number in the interval $[0, 1)$. Unlike Huffman coding, which assigns a fixed number of bits per symbol, arithmetic coding assigns *fractional* bits by encoding the entire sequence into a progressively refined interval.

It is widely used in:

- JPEG2000
- H.264 / H.265 video coding (CABAC)

- Modern image codecs such as AVIF, WebP, HEIC

Arithmetic coding achieves compression rates extremely close to the entropy limit.

16.2 Basic Concept

Given a source alphabet:

$$\mathcal{A} = \{a_1, a_2, \dots, a_n\}$$

with probabilities:

$$P = \{p_1, p_2, \dots, p_n\}$$

We construct the cumulative distribution:

$$C_0 = 0, \quad C_i = \sum_{k=1}^i p_k$$

Each symbol a_i corresponds to the subinterval:

$$[C_{i-1}, C_i)$$

16.3 Encoding Procedure

Arithmetic coding maintains a narrowing interval:

$$[L, U)$$

initially:

$$L = 0, \quad U = 1.$$

For each symbol s :

1. Compute current range:

$$R = U - L$$

2. Set:

$$U \leftarrow L + R \cdot C_{s,\text{high}}$$

$$L \leftarrow L + R \cdot C_{s,\text{low}}$$

After encoding all symbols, any number in the final interval uniquely represents the message.

16.4 Decoding Procedure

Decoder receives a number $x \in [L, U)$. At each step:

1. Determine which subinterval contains x .
2. Output the corresponding symbol.
3. Update interval using same formulas as encoder.
4. Repeat until message length is reached.

16.5 Advantages Over Huffman Coding

- No requirement for symbol code lengths to be integers.
- Compression efficiency near entropy:

$$L \approx H(X)$$

- Easily extends to:
 - adaptive models
 - context-based probability estimation
 - high-order Markov models
- Crucial for modern codecs handling complex symbol statistics.

16.6 Renormalization (Practical Implementation)

In practice, intervals become extremely small. To avoid underflow:

- Range is rescaled when L and U fall into the same half or quarter.
- Bits are emitted progressively.
- This is done using integer arithmetic.

This is the basis of:

- MQ-coder (JPEG2000)
- CABAC (H.264/AVC, H.265/HEVC)

16.7 Example Problem 1: Arithmetic Encoding

Problem

Symbols and probabilities:

$$A : 0.5, \quad B : 0.3, \quad C : 0.2$$

Cumulative intervals:

$$A : [0, 0.5), \quad B : [0.5, 0.8), \quad C : [0.8, 1.0)$$

Encode the message:

“ABC”

Solution

Initial: $L = 0, U = 1$

Step 1: Encode A

$$R = 1 - 0 = 1$$

$$L = 0 + 1 \cdot 0 = 0, \quad U = 0 + 1 \cdot 0.5 = 0.5$$

New interval: $[0, 0.5)$

Step 2: Encode B

$$R = 0.5 - 0 = 0.5$$

$$L = 0 + 0.5 \cdot 0.5 = 0.25$$

$$U = 0 + 0.5 \cdot 0.8 = 0.40$$

New interval: $[0.25, 0.40)$

Step 3: Encode C

$$R = 0.40 - 0.25 = 0.15$$

$$L = 0.25 + 0.15 \cdot 0.8 = 0.37$$

$$U = 0.25 + 0.15 \cdot 1.0 = 0.40$$

Final interval:

$$[0.37, 0.40)$$

Any number in this interval—commonly the lower bound:

$$\boxed{0.37}$$

represents the full message “ABC”.

16.8 Example Problem 2: Arithmetic Decoding

Problem

Use the same probability model as above. Decode the number:

$$x = 0.381$$

Message length = 3 symbols.

Solution

Initial interval: $[0, 1)$

Step 1: Find symbol for $x = 0.381$

Intervals:

$$A : [0, 0.5), \quad B : [0.5, 0.8), \quad C : [0.8, 1)$$

Since $0.381 \in A$, output:

A

Update interval to $[0, 0.5)$.

Step 2: Decode within interval $[0, 0.5)$

Normalize:

$$x' = \frac{x - 0}{0.5 - 0} = 0.381/0.5 = 0.762$$

Check subintervals of A/B/C:

$$A : [0, 0.5), \quad B : [0.5, 0.8), \quad C : [0.8, 1)$$

Since $0.762 \in B$, symbol = B.

Update interval to:

$$[0.25, 0.40)$$

Step 3: Decode last symbol

Normalize:

$$x'' = \frac{0.381 - 0.25}{0.40 - 0.25} = \frac{0.131}{0.15} \approx 0.873$$

$0.873 \in C$ interval.

Output: C

Final Decoded Message

ABC

16.9 Summary

- Arithmetic coding represents a sequence as a progressively shrinking interval.
- Achieves compression near the entropy limit.
- More powerful than Huffman coding for non-integer probabilities.
- Forms the basis of modern image/video compression systems.
- Encoding = refining $[L, U)$ interval; decoding = locating x within nested intervals.

17 Golomb–Rice Coding

17.1 Introduction

Golomb coding is a lossless entropy coding method designed for sources whose symbol distribution resembles a **geometric distribution**:

$$P(X = k) = (1 - p)^k p, \quad k = 0, 1, 2, \dots$$

Golomb codes are optimal for such distributions and are widely used in:

- Lossless image compression (JPEG-LS)
- Audio compression (FLAC residual coding)
- Predictive coding of images (prediction errors often geometric/Laplacian)

A special case of Golomb coding, called **Rice coding**, occurs when the divisor parameter is a power of two.

17.2 Golomb Coding Overview

Golomb coding uses a parameter m , chosen based on the statistical distribution of the data. For a non-negative integer N , compute:

$$q = \left\lfloor \frac{N}{m} \right\rfloor, \quad r = N \bmod m$$

The Golomb code for N is:

$$\text{code}(N) = \underbrace{11 \dots 10}_{q \text{ times}} \parallel \text{binary_code}(r)$$

The first part is unary coding of q , followed by a truncated binary encoding of the remainder r .

17.3 Truncated Binary Encoding of the Remainder

Let:

$$b = \lceil \log_2 m \rceil$$

and:

$$t = 2^b - m$$

Then encoding of remainder r is:

$$\text{binary_code}(r) = \begin{cases} \text{binary}(r), & r < t \\ \text{binary}(r + t), & r \geq t \end{cases}$$

17.4 Decoding

Given a Golomb code:

1. Count number of leading 1s until the first 0 $\rightarrow$ this gives q .
2. Decode the remaining bits to obtain r (undo truncated binary).
3. Reconstruct value:

$$N = qm + r$$

17.5 Choosing the Parameter m

For geometric distributions:

$$m \approx -\frac{1}{\ln(1-p)}$$

In image compression (JPEG-LS), the optimal m is computed adaptively from the variance of prediction errors.

17.6 Rice Coding (Special Case)

Rice coding is a simplified Golomb code where:

$$m = 2^k$$

Thus:

$$q = N \gg k \quad (\text{right shift})$$

$$r = N \& (2^k - 1)$$

Binary representation of r is simply the k -bit binary number.

This makes Rice coding extremely efficient for hardware and software implementations.

17.7 Example Problem 1: Golomb Encoding

Problem

Encode the integer $N = 37$ using Golomb coding with parameter $m = 10$.

Solution

Compute:

$$q = \left\lfloor \frac{37}{10} \right\rfloor = 3, \quad r = 37 \bmod 10 = 7$$

Unary code for $q = 3$:

1110

Now encode remainder:

$$b = \lceil \log_2 10 \rceil = 4, \quad t = 2^4 - 10 = 6$$

Since $r = 7 \geq t = 6$, encode:

$$r + t = 7 + 6 = 13$$

Binary(13) in 4 bits:

1101

Final Golomb Code

11101101

17.8 Example Problem 2: Rice Coding

Problem

Encode the integer sequence using Rice coding with parameter $k = 2$ (i.e., $m = 4$):

$$N = 6$$

Solution

Compute:

$$q = N \gg k = 6 \gg 2 = 1$$

$$r = N \& (2^k - 1) = 6 \& 3 = 6 \bmod 4 = 2$$

Unary code of $q = 1$:

10

Binary representation of $r = 2$ with $k = 2$ bits:

10

Final Rice Code

10 10

17.9 Why Golomb–Rice Coding Works for Images

In predictive image coding (JPEG-LS), the residuals:

$$e(i, j) = x(i, j) - \hat{x}(i, j)$$

tend to follow a Laplacian distribution centered at zero. After mapping to non-negative values using the standard “zig-zag” mapping:

$$0, 1, -1, 2, -2, 3, -3, \dots \Rightarrow 0, 1, 2, 3, 4, 5, 6, \dots$$

the distribution becomes geometric-like:

$$P(N = k) \propto (1 - p)^k$$

Golomb and Rice coding are **optimal** for such distributions.

17.10 Summary

- Golomb codes optimally compress geometric distributions.
- Code for integer N : unary(q) + truncated-binary(r).
- Rice coding is a special case with $m = 2^k$, making encoding extremely efficient.
- Widely used in predictive lossless image compression (JPEG-LS).
- Two steps encode each value: quotient (run-length-like) and remainder (binary-coded).

18 Haar Wavelet Transform

18.1 Introduction

Wavelet transforms provide a multi-resolution representation of signals and images. Unlike the Fourier transform, which represents global frequency content, wavelets provide:

- local frequency information,

- multi-scale decomposition,
- sparse representations useful for compression.

The **Haar wavelet** is the simplest orthonormal wavelet, introduced by Alfréd Haar in 1910. It is widely used in:

- image compression (JPEG2000 preliminaries),
- denoising,
- feature extraction,
- multi-resolution analysis.

18.2 Haar Scaling and Wavelet Functions

The Haar scaling function is:

$$\phi(t) = \begin{cases} 1, & 0 \leq t < 1, \\ 0, & \text{otherwise.} \end{cases}$$

The wavelet function is:

$$\psi(t) = \begin{cases} 1, & 0 \leq t < \frac{1}{2}, \\ -1, & \frac{1}{2} \leq t < 1, \\ 0, & \text{otherwise.} \end{cases}$$

These functions generate the Haar basis for $L^2(\mathbb{R})$.

18.3 1D Haar Transform

Given a discrete signal $x = [x_1, x_2, \dots, x_N]$, with N a power of 2, the 1-level Haar transform is defined as:

$$A_k = \frac{x_{2k-1} + x_{2k}}{2}, \quad D_k = \frac{x_{2k-1} - x_{2k}}{2}, \quad k = 1, \dots, N/2.$$

$$\text{Output} = [A_1, A_2, \dots, A_{N/2}, D_1, D_2, \dots, D_{N/2}]$$

18.4 Inverse 1D Haar Transform

$$x_{2k-1} = A_k + D_k, \quad x_{2k} = A_k - D_k.$$

This reconstruction is exact since Haar is orthonormal.

18.5 2D Haar Transform (Images)

The 2D Haar transform applies the 1D transform:

1. to each **row**,
2. then to each **column**.

The four resulting subbands are:

$$\begin{bmatrix} LL & LH \\ HL & HH \end{bmatrix}$$

Where:

- LL = low-pass rows + low-pass columns,
- LH = low-pass rows + high-pass columns,
- HL = high-pass rows + low-pass columns,
- HH = high-pass rows + high-pass columns.

18.6 Wavelet-Based Image Compression

Compression occurs because:

- Most energy is in the LL band,
- Detail bands (LH , HL , HH) often contain small coefficients,
- Thresholding those coefficients introduces minimal perceptual distortion.

Thresholding rule:

$$c' = \begin{cases} c, & |c| \geq T, \\ 0, & |c| < T. \end{cases}$$

The number of non-zero coefficients measures compression efficiency.

Exam-Level Problems With Full Solutions

Problem 1: 1D Haar Transform

Compute the 1-level Haar transform of:

$$x = [8, 6, 4, 2].$$

Solution

Approximation coefficients:

$$A_1 = \frac{8+6}{2} = 7, \quad A_2 = \frac{4+2}{2} = 3.$$

Detail coefficients:

$$D_1 = \frac{8-6}{2} = 1, \quad D_2 = \frac{4-2}{2} = 1.$$

$$[7, 3, 1, 1]$$

Problem 2: 2D Haar Transform of Image Block

Given:

$$I = \begin{bmatrix} 4 & 4 & 8 & 8 \\ 4 & 4 & 8 & 8 \\ 10 & 10 & 6 & 6 \\ 10 & 10 & 6 & 6 \end{bmatrix}$$

Compute 1-level 2D Haar transform.

Solution

Step 1: Row transforms Row averages/differences:

Row 1:

$$[4, 4, 8, 8] \rightarrow [4, 8 \mid 0, 0]$$

Row 2:

$$[4, 4, 8, 8] \rightarrow [4, 8 \mid 0, 0]$$

Row 3:

$$[10, 10, 6, 6] \rightarrow [10, 6 \mid 0, 0]$$

Row 4:

$$[10, 10, 6, 6] \rightarrow [10, 6 \mid 0, 0]$$

Matrix now:

$$\begin{bmatrix} 4 & 8 & 0 & 0 \\ 4 & 8 & 0 & 0 \\ 10 & 6 & 0 & 0 \\ 10 & 6 & 0 & 0 \end{bmatrix}$$

Step 2: Column transforms on left two columns Column 1:

$$[4, 4, 10, 10] \rightarrow [4, 10 \mid 0, 0]$$

Column 2:

$$[8, 8, 6, 6] \rightarrow [8, 6 \mid 0, 0]$$

Final Subbands

$$LL = \begin{bmatrix} 4 & 8 \\ 10 & 6 \end{bmatrix}, \quad LH = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix},$$

$$HL = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad HH = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Problem 3: Inverse 1D Haar Transform

Given:

$$A_1 = 7, A_2 = 3, D_1 = 1, D_2 = -1$$

Reconstruct signal.

Solution

$$x_1 = A_1 + D_1 = 8, \quad x_2 = A_1 - D_1 = 6$$

$$x_3 = A_2 + D_2 = 2, \quad x_4 = A_2 - D_2 = 4$$

$$\boxed{[8, 6, 2, 4]}$$

Problem 4: Wavelet Compression (Thresholding)

Given 2D coefficient matrix:

$$C = \begin{bmatrix} 18 & 5 & -3 & 1 \\ 2 & -1 & 0 & 0 \\ -4 & 1 & 2 & -1 \\ 3 & 0 & 0 & 0 \end{bmatrix}$$

Threshold $T = 2$.

Solution

Apply hard thresholding:

Values with magnitude < 2 become 0.

$$C' = \begin{bmatrix} 18 & 5 & -3 & 0 \\ 2 & 0 & 0 & 0 \\ -4 & 0 & 2 & -1 \\ 3 & 0 & 0 & 0 \end{bmatrix}$$

Count nonzero before:

$$NZ_{\text{orig}} = 11$$

Count nonzero after:

$$NZ_{\text{new}} = 8$$

Compression ratio:

$$CR = \frac{11}{8} = 1.375$$

—

Problem 5: 2-Level Haar Transform (1D)

Signal:

$$x = [12, 8, 4, 4, 10, 6, 2, 2]$$

Solution

Level 1 Pairs:

$$A_1 = 10, D_1 = 2$$

$$A_2 = 4, D_2 = 0$$

$$A_3 = 8, D_3 = 2$$

$$A_4 = 2, D_4 = 0$$

$$A_1 = [10, 4, 8, 2], \quad D_1 = [2, 0, 2, 0]$$

Level 2 on A_1 Pairs:

$$A_2^{(1)} = 7, D_2^{(1)} = 3$$

$$A_2^{(2)} = 5, D_2^{(2)} = 3$$

Thus:

$$A_2 = [7, 5], \quad D_2 = [3, 3]$$

Final 2-Level Output

$$A_2 = [7, 5], D_2 = [3, 3], D_1 = [2, 0, 2, 0]$$

$$[7, 5, 3, 3, 2, 0, 2, 0]$$

Summary

- Haar transform performs averaging and differencing operations.
- 2D Haar works by applying 1D transform to rows and columns.
- LL preserves most information, LH/HL/HH capture edges.
- Thresholding wavelet coefficients yields compression.
- Multi-level transforms reveal progressively coarser approximations.

19 Wavelet Compression Using the LL Band

19.1 Introduction

Wavelet-based image compression exploits the fact that natural images concentrate most of their visual energy in the low-frequency components. After a 2D wavelet transform, the image is decomposed into four subbands:

$$\begin{bmatrix} LL & LH \\ HL & HH \end{bmatrix}$$

where:

- LL contains coarse approximation (low frequencies),
- LH contains horizontal edges,
- HL contains vertical edges,
- HH contains diagonal edges.

The LL subband retains the majority of the signal energy and is recursively decomposed in multi-level wavelet transforms (as in JPEG2000).

19.2 Energy Compaction Property

For many natural signals, wavelet transforms achieve **sparse representations**. Mathematically:

$$\|LL\|_2^2 \gg \|LH\|_2^2 + \|HL\|_2^2 + \|HH\|_2^2.$$

That is, most of the energy lies in the approximation band. This allows:

- aggressive quantization of detail bands,
- preservation of perceptually important low-frequency content,
- high compression ratios with minimal visual distortion.

19.3 Compression Pipeline

A typical wavelet-based compression system performs:

1. 2D wavelet transform

$$I \xrightarrow{W} \{LL, LH, HL, HH\}.$$

2. Quantization For detail coefficients:

$$c' = \text{sign}(c) \cdot \max(|c| - T, 0)$$

or:

$$c' = \begin{cases} 0, & |c| < T, \\ \text{round}(c/Q), & |c| \geq T. \end{cases}$$

3. Entropy coding (Huffman, arithmetic, or Golomb–Rice).

4. Reconstruction

$$\{LL', LH', HL', HH'\} \xrightarrow{W^{-1}} \hat{I}.$$

19.4 Multi-Level LL Decomposition

Recursive decomposition applies the wavelet transform to the LL subband:

$$LL_0 = I$$

$$LL_1, LH_1, HL_1, HH_1 = W(LL_0)$$

$$LL_2, LH_2, HL_2, HH_2 = W(LL_1)$$

After L levels:

$$\text{Output} = \{LL_L, LH_L, HL_L, HH_L, \dots, LH_1, HL_1, HH_1\}.$$

Only LL_L needs to be stored with high fidelity; the rest can be heavily quantized.

19.5 Rate–Distortion Considerations

For a given bitrate, optimal bit allocation follows:

$$b_{LL} > b_{LH} \approx b_{HL} > b_{HH}.$$

The distortion D is:

$$D = \sum_{\text{subbands}} w_s \cdot E[(c_s - \hat{c}_s)^2],$$

where w_s accounts for each band's contribution to image quality.

19.6 Compression Ratio Calculation

If the original image has N coefficients and the compressed representation contains K non-zero or entropy-coded coefficients:

$$\text{CR} = \frac{N}{K}.$$



Exam-Level Problems With Complete Solutions

Problem 1: Energy Compaction in Wavelet Transform

Given the subband energies:

$$E_{LL} = 8200, \quad E_{LH} = 500, \quad E_{HL} = 620, \quad E_{HH} = 300.$$

Compute the percentage of total energy in the LL band.

Solution

Total energy:

$$E_{\text{tot}} = 8200 + 500 + 620 + 300 = 9620.$$

$$\%LL = \frac{8200}{9620} \times 100 = 85.2\%.$$

85.2% of the energy lies in the LL band.

Problem 2: Thresholding for Compression

Given detail coefficients:

$$D = [3, -1, 0, 2, 7, -3, 1]$$

Apply hard thresholding with $T = 3$.

Solution

Rule:

$$c' = \begin{cases} 0, & |c| < 3 \\ c, & |c| \geq 3 \end{cases}$$

Apply:

$$[3, 0, 0, 0, 7, -3, 0]$$

Number of nonzero coefficients:

$$K = 3$$

Original count:

$$N = 7$$

Compression ratio:

$$CR = \frac{7}{3} = 2.33.$$

Problem 3: Multi-Level LL Decomposition

An image is decomposed using two-level Haar transform:

$$LL_2 = \begin{bmatrix} 50 & 60 \\ 55 & 65 \end{bmatrix}$$

LH_2, HL_2, HH_2 have small values and are heavily quantized to zero.
Explain qualitatively what reconstruction looks like.

Solution

Since:

- LL_2 contains coarse approximation of the image,
- detail coefficients at level 2 are zero,
- detail coefficients at level 1 are small or zero,

Reconstruction approximates a blurred/smoothed image with dominant low-frequency components preserved.

Reconstruction looks like a low-resolution, smoothed version of the original image.

Problem 4: 2D Wavelet Coefficient Storage

A 256×256 image undergoes 3-level wavelet transform.
Compute sizes of subbands.

Solution

Level 1:

$$LL_1 : 128 \times 128, \quad LH_1 : 128 \times 128, \quad HL_1 : 128 \times 128, \quad HH_1 : 128 \times 128.$$

Level 2 (applied to LL_1):

$$LL_2 : 64 \times 64, \quad LH_2 : 64 \times 64, \quad HL_2 : 64 \times 64, \quad HH_2 : 64 \times 64.$$

Level 3:

$$LL_3 : 32 \times 32, \quad LH_3 : 32 \times 32, \quad HL_3 : 32 \times 32, \quad HH_3 : 32 \times 32.$$

Total coefficient count:

$$32^2 + 3 \cdot 64^2 + 3 \cdot 128^2 = 1024 + 12288 + 49152 = 62464,$$

which equals $256^2 = 65536$ (after rounding behavior differences).

Problem 5: Compression Using Only LL Band

A 512×512 image is compressed by storing:

- the LL_2 band (size 128×128),
- 20% of the largest detail coefficients from all other bands.

Compute the fraction of coefficients retained.

Solution

Total coefficients:

$$= 512^2 = 262144.$$

LL_2 size:

$$128^2 = 16384.$$

Detail coefficients total:

$$262144 - 16384 = 245760.$$

We keep 20% of them:

$$0.20 \times 245760 = 49152.$$

Total kept:

$$16384 + 49152 = 65536.$$

Fraction retained:

$$\frac{65536}{262144} = 0.25.$$

25% of coefficients retained.

Summary

- Wavelet compression relies on the dominance of the LL band.
- Detail bands can be quantized or thresholded with minimal perceptual loss.
- Multi-level decomposition increases compression efficiency.
- Energy largely concentrates in low-frequency wavelet components.
- Compression ratio improves as more detail coefficients are removed.

20 Predictive Coding (DPCM and JPEG-LS / LOCO-I)

20.1 Introduction

Predictive coding exploits statistical redundancy in images by predicting the value of each pixel from its neighbors. Rather than encoding raw pixel intensities, we encode the **prediction error**, which typically has a sharply peaked distribution around zero and is more compressible.

Predictive coding is central to image compression algorithms including:

- Differential Pulse Code Modulation (DPCM),
- Lossless JPEG (obsolete),
- JPEG-LS (LOCO-I) — the industry standard for lossless image compression,
- High-quality predictive codecs (e.g., FLIF).

The general predictive model is:

$$\hat{x}(m, n) = P(\mathcal{N}) \quad \text{predictor from neighborhood}$$

$$e(m, n) = x(m, n) - \hat{x}(m, n)$$

Then we encode $e(m, n)$ instead of $x(m, n)$.

During decoding:

$$\hat{x}(m, n) = \hat{x}(m, n) + e(m, n)$$

This requires only that the encoder and decoder use the same predictor.

20.2 DPCM (Differential Pulse Code Modulation)

DPCM uses linear predictors based on spatial neighbors. Common linear predictors:

1. First-order predictor

$$\hat{x}(m, n) = x(m, n - 1)$$

2. Two-dimensional linear predictor

$$\hat{x}(m, n) = ax(m, n - 1) + bx(m - 1, n) + cx(m - 1, n - 1)$$

Typical JPEG predictor (lossless mode):

$$\hat{x}(m, n) = x(m, n - 1) + x(m - 1, n) - x(m - 1, n - 1)$$

20.3 Prediction Error

$$e(m, n) = x(m, n) - \hat{x}(m, n)$$

If the predictor is good, $e(m, n)$ has:

- small magnitude,
- Laplacian or sharply peaked distribution,
- low entropy.

This reduces average code length when entropy coding is applied.

20.4 Quantization in Predictive Coding

For **lossy** predictive coding:

$$\hat{e}(m, n) = Q(e(m, n))$$

Decoder reconstructs:

$$\hat{x}(m, n) = \hat{x}(m, n) + \hat{e}(m, n)$$

Quantization introduces distortion but reduces entropy dramatically.

20.5 JPEG-LS (LOCO-I): Nonlinear Predictive Coding

JPEG-LS uses the **Median Edge Detector (MED)** predictor.

Let pixels be defined as:

$$\begin{aligned} C &= x(m, n-1), & B &= x(m-1, n) \\ A &= x(m-1, n-1) \end{aligned}$$

$$\begin{array}{cc} A & B \\ C & X \end{array}$$

The MED predictor is:

$$\hat{x} = \begin{cases} \min(B, C), & A \geq \max(B, C), \\ \max(B, C), & A \leq \min(B, C), \\ B + C - A, & \text{otherwise.} \end{cases}$$

Interpretation

The MED predictor adapts to local gradients:

- If A is very large (upper-left corner is bright), it predicts the minimum of neighbors: indicating a downward edge.
- If A is very small, predicts the maximum: indicating an upward edge.
- Otherwise, uses a linear predictor $B + C - A$ which estimates smooth regions.

This nonlinearity allows JPEG-LS to outperform older linear predictors.

20.6 Run Mode in JPEG-LS

When the image contains long flat regions:

$$x(m, n) \approx x(m, n - 1) \approx x(m, n - 2) \approx \dots$$

JPEG-LS switches to run-length mode:

- Encode the length of the flat run.
- Encode a terminating sample in regular mode.

This exploits the fact that natural images often contain smooth areas.

20.7 Context Modeling (LOCO-I)

Prediction residuals are not encoded uniformly. Instead, JPEG-LS uses 365 contexts derived from local gradients:

$$g_1 = B - A, \quad g_2 = A - C, \quad g_3 = C - B$$

Each context k stores:

- bias,
- variance-like estimate,
- Golomb parameter for entropy coding.

This makes JPEG-LS nearly optimal for lossless coding.

20.8 Entropy Coding in Predictive Coders

Prediction error $e(m, n)$ is entropy-coded using ****Golomb-Rice coding****, ideal for Laplacian distributions.

$$P(e) \approx \frac{1}{2} \alpha e^{-\alpha|e|}$$

Small errors occur frequently, so Rice codes are efficient.

20.9 Rate–Distortion

For lossy predictive coding:

$$D = E[(x - \hat{x})^2]$$

Bitrate increases with:

$$R \approx H(e_Q)$$

Optimal quantizer step size balances R and D .

Exam-Level Problems With Solutions

Problem 1: MED Prediction

Given:

$$A = 120, B = 100, C = 90.$$

Compute the MED prediction.

Solution

Check conditions:

$$A = 120 \geq \max(B, C) = 100 \Rightarrow \hat{x} = \min(B, C) = 90.$$

$$\boxed{\hat{x} = 90}$$

Problem 2: Compute Prediction Error

Given:

$$x(m, n) = 110, \quad \hat{x} = 100.$$

$$e = 110 - 100 = 10.$$

If quantizer $Q(e) = \text{round}(e/4)$:

$$\hat{e} = 10/4 = 2.5 \rightarrow 3.$$

Decoder reconstructs:

$$\hat{x} = 100 + 3 = 103.$$

$$\boxed{\text{Reconstructed value} = 103}$$

Problem 3: DPCM Linear Prediction

Image row:

$$[100, 104, 108, 130]$$

Predictor:

$$\hat{x}(n) = x(n-1)$$

Compute prediction errors.

Solution

$$\begin{aligned}\hat{x}(2) &= 100, & e &= 104 - 100 = 4 \\ \hat{x}(3) &= 104, & e &= 108 - 104 = 4 \\ \hat{x}(4) &= 108, & e &= 130 - 108 = 22\end{aligned}$$

$$e = [4, 4, 22]$$

Problem 4: JPEG-LS Gradient Context

Given:

$$A = 50, \quad B = 60, \quad C = 55.$$

Compute the gradients:

$$g_1 = B - A = 10$$

$$g_2 = A - C = -5$$

$$g_3 = C - B = -5$$

This defines the context used for Golomb coding.

$$(g_1, g_2, g_3) = (10, -5, -5)$$

Problem 5: Compression Gain from Prediction

Original pixel entropy:

$$H(X) = 7.8 \text{ bits}$$

Residual entropy:

$$H(E) = 4.1 \text{ bits}$$

Compute compression gain:

$$G = \frac{H(X)}{H(E)} = \frac{7.8}{4.1} = 1.90.$$

$$\text{Prediction reduces coding cost by a factor of 1.9}$$

Summary

- DPCM uses linear prediction; JPEG-LS uses nonlinear MED prediction.
- Prediction reduces entropy of image data dramatically.
- JPEG-LS uses context modeling and Golomb-Rice coding for optimal performance.
- Run mode efficiently handles flat regions.
- Predictive coding is essential for both lossless and lossy compression.